数据结构

《数据结构（用面向对象方法和C++语言描述） 第2版》殷人昆 清华大学出版社 ISBN 978-7-302-14811-1

第1章、数据结构概述

1. 数据的逻辑结构： 1）线性结构：直接存取-数组，顺序存取-链表，字典结构-hash；
   2）非线性结构：层次结构-树，群关系-图&集合；
2. 数据的存储结构：1）顺序存储；2）链接存储（指针，引用）；3）索引存储（附加索引表建立key与地址的映射）；4）散列存储（由key可直接计算地址）；
3. 算法是一个有限的指令集合，为解决某一特定任务规定的运算序列；有输入，有输出，确定性，有穷性，可行性；
4. 算法的性能标准：正确性，可使用性，可读性，效率，健壮性，简单性；后期测试，前期估计；空间复杂度，时间复杂度；渐进分析，最好，最坏，平均情况：O-描述上限，Ω-描述下限，Θ-上下限相等；
   $O(1)< O(\log n)< O(n)<O(n*\log n)<O(n^2) <O(n^3) <O(2^n) <O(n!) <O(n^n)$；
   $O(\cdot)$只是算法随规模n变化的**趋势**，但n较小时则不一定符合该趋势，如n=3时，反而有$n^3>2^n> n!$；

第2章、线性表

1. 一般而言，线性表的元素数据类型是相同的；广义表则允许元素有不同的数据类型；线性表有两种**存储**表示：顺序存储和链式存储；
2. 顺序表：元素的逻辑顺序与物理存放顺序一致；可顺序访问也可随机访问，时间复杂度为O(1)；插入和删除任意元素需移动后续所有元素，效率较低，时间复杂度为O(n)；
3. 单链表：长度可以方便地扩充；只能顺序访问，遍历效率低，时间复杂度为O(n)；插入和删除元素只需修改相关结点的指针，较高效，**但定位该元素需要顺序查找，总的时间复杂度仍为O(n)**；
4. 带头结点的单链表；循环链表（首尾相接），应用 — 约瑟夫问题；双向链表；
5. 静态链表：数组中每个元素有一个link域，无需改变元素位置，改变link域即可改变元素的逻辑顺序；与单链表性能相似，但link域不是物理地址，只是元素在数组中的序号；

第3章、栈和队列

1. 栈Stack是只允许在末端插入和删除的线性表，LIFO；栈可以由顺序表，也可以由链表构成；栈的基本操作：pop，push；若由单链表构成栈，则pop时必须删除结点，否则会造成内存泄漏；用顺序表存储栈时，有的将top指向真实的栈顶，**有的**则将top指向真实栈顶的下一个位置；
2. 栈的应用
   1) 括号匹配 — 遇到左括号入栈，遇到右括号则将左括号弹出栈，若最后栈为空说明左右括号匹配；每种括号一个栈，即可匹配有多种括号的表达式（允许交叉），若不允许交叉，只用一个栈即可；
   2) 后缀表达式的计算 — 又称逆波兰式，RPN，栈内元素是操作**数**，顺序扫描RPN表达式，若是操作数则入栈，若为（双目）操作符op，则连续两次退栈，得到X，Y，计算X op Y，并把结果入栈；表达式扫描完成后，栈顶就是结果；
   3) 中缀表达式转为后缀表达式 — 栈内元素是操作**符**后缀表达式，需考虑各操作符的**栈内优先级ISP**和**栈外优先级ICP**，主要是左右括号的优先级变化；顺序扫描中缀表达式，若为操作数则直接输出，若为操作符op，判断其栈外优先级与栈顶操作符top的栈内优先级，若op.ICP>top.ISP，则将op入栈，继续扫描；若op.ICP<top.ISP，则退栈并输出top，op继续与新的top比较，直到遇到较小ISP或栈空，将op入栈，继续扫描；若op.ICP==top.ISP，退栈但不输出，继续扫描；

操作符	#	(	* / %	+ -	)
ISP	0	1	5	3	6
ICP	0	6	4	2	1

   4) 用栈实现函数调用，递归，显式递归（回溯法）；
3. 队列是只允许在**末端插入，在首端删除**的线性表，FIFO；
4. 循环队列（队尾指向的是下一个末端的空位，即队列至少有一个空闲位）：
   出队 front=(front+1)%size;　　入队 rear=(rear+1)%size;
   空　front==rear;　　满　(rear+1)%size==front;
   元素个数 (rear – front + size)%size;

5．队列应用

1) 计算二项式的系数（杨辉三角） — 输出当前行时，就可以计算出下一行并入队，各行间需插入一个 0；

2) 电路布线，迷宫 — 用队列跟踪那些已经被标记，但其相邻点尚未标记的点；全部点都标记后回溯；

6．优先级队列：**不是** FIFO 队列，每次从队首取得的是优先级最大的元素；可以用一般队列实现，在插入或删除后需调整队列，将优先级最大的元素移动到队首；也可用堆（heap）实现；

第4章、数组、串和广义表

1．多维数组，行优先存储；特殊矩阵（对称阵，上、下三角阵等）的存储；

2．稀疏矩阵(sparse matrix)：三元组存储（行，列，值）；正交链存储；

3．字符串；子串的模式匹配：朴素算法，有回溯，O(n*m)，适合较短的字符串；**KMP 算法**，无回溯，需计算 next 函数以反映子串自身的模式，O(n+m)；Boyer-Moore 算法：从子串尾部开始比较；

4．广义表，递归定义的，允许表中有表，类似 LISP 中的 list；有序，有长度，深度，递归的；

第5章、树

注：满二叉树；完全二叉树，完全二叉树的叶结点可能在 k 和 k-1 层；二叉树的性质

1) 二叉树的第 i 层**最多**有 2^{i-1} 个结点，注意：以根结点所在的层为第 **1** 层；

2) 深度为 k 的二叉树最**少**有 k 个结点，最**多**有 2^k-1 个结点；

3) 对任一棵非空二叉树，如果叶结点数为 n_0，度为 2 的非叶结点数为 n_2，则 $n_0=n_2+1$；
边数$e=n-1=n_0+n_1+n_2-1$，又有$e=n_1+2\cdot n_2$，由此即可得上面的性质；

4) 有 n 个结点的**完全二叉树**的深度为 ceil(log2(n+1))；

5) 若将**完全二叉树**从上到下，从左到右顺序存储，结点编号 1~n，则有

a.若结点编号 i==1，则为二叉树的根结点；若 i>1，则其父结点为 floor(i/2)；

b.若 2*i <=n，则结点 i 的**左**孩子为结点 2*i ；否则没有左孩子，当然也就没有右孩子；

c.若 2*i+1<=n，则结点 i 的**右**孩子为结点 2*i+1；否则没有右孩子；

d.若结点 i 为**奇**数且≠1，则它处于**右**兄弟位置；

e.若结点 i 为**偶**数，则它处于**左**兄弟位置；

6) 若**完全二叉树**结点编号 0~n-1，则

a.若结点编号 i==0，则为二叉树的根结点；若 i>0，则其父结点为 floor((i-1)/2)；

b.若 2*i+1<=n-1，则结点 i 的**左**孩子为结点 2*i+1；否则没有左孩子，当然也就没有右孩子；

c.若 2*i+2<=n-1，则结点 i 的**右**孩子为结点 2*i+2；否则没有右孩子；

d.若结点 i 为**偶**数且≠0，则它处于**右**兄弟位置；

e.若结点 i 为**奇**数，则它处于**左**兄弟位置；

若 u 为 unsigned int，则 floor(u/2)=u/2，ceil(u/2)=(u+1)/2=floor((u+1)/2)；ceil(u/2)≠u/2+1；

ceil(log2(u+1))是 u 最高非 0 位（左边第一个 1）的位数（从右向左数，依次为 1~32 位），如 8 用二进制表示为 1000，最高非 0 位在第 4 位，得 ceil(log2(9))=4，可通过连续左移实现；

搞不清上面关系时画一个简单的二叉树带入比较一下即可很快得出结论；

n 个结点能构成二叉树的个数为$C_{2n}^n/(n+1)$，称为 catalan 函数，这也是 n 个编号的元素以**某种**顺序进栈，任意顺序出栈（即进出栈可以交替进行），得到的元素排列的个数；

2．二叉树的存储

1) 完全二叉树可以从上到下，从左到右顺序存储在一维数组，但注意数组应从 1 开始编号，以方便相关计算；

2) **一般**二叉树可预留空位后按完全二叉树存储；也可用二叉链表存储；为了方便寻找父结点，增加一个父指针域，成为三叉链表；二叉和三叉链表都可用静态链表结构；亦可用**仅有父指针**的静态链表来存储；

3．二叉树的遍历

1) 递归算法：前序 VLR（树的复制，相等，广义表格式打印），中序 LVR，后序 LRV（结点计数，高度）；

```
void Traverse(BinTree & tree, void (* visit)(BinTree & node)){
    if(tree!=NULL){
        Traverse(tree.left,  visit);
        visit(tree);                    //VLR, LVR,LRV 差别只在 visit 的顺序；
        Traverse(tree.right, visit);
    }
}
```

2) 非递归算法，需要一个栈实现显式递归，后序(LRV)还需要增加辅助标识，以区分 L,R；
3) 由给定的前序（后序）和中序序列能唯一确定一棵二叉树；
4) 层序遍历（广度优先），需要一个队列，在访问某一层时，将下一层的结点入队；

```
void LevelTraverse (BinTree& tree, void (* visit)(BinTree & node)){
    Quene* q= new Quene();
    q->EnQuene(tree);
    while(!q->IsEmpty()){
        tree=q->DeQuene();    visit(tree);
        if(tree.left !=NULL) q->EnQuene(tree.left);
        if(tree.right!=NULL) q->EnQuene(tree.right);
    }
    delete q;
}
```

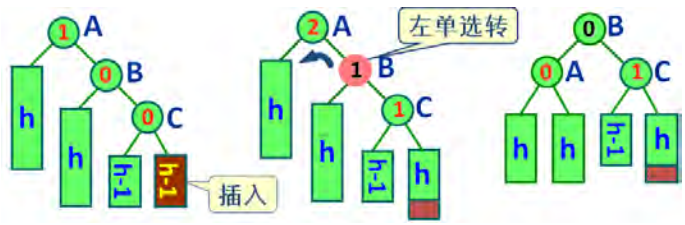
4. 线索二叉树：n 个结点的二叉链表中有 n+1 个空指针，可以利用其指向前驱或后继结点，称为线索，这时**每个**指针域需附加一个标志域，以区分指针指向的是子树还是线索；二叉树的不同遍历方法对应了不同的线性序列，线索即指明了这种线性序列的前驱/后继关系；
   线索二叉树的遍历；建立线索的方法；插入和删除结点的方法；中序线索实现前序和后序遍历；
5. 树的度是所有结点的度的最大值；一般树的存储：1）广义表（即链表）；2）父指针，父结点和子结点是 1:N 的关系，每个子结点只需要一个父指针；3）孩子链表，每个结点有一个链表，保存所有孩子结点，还可以与父指针法结合起来；4）孩子-兄弟链表，又称长子-兄弟链表，是一种二叉树表示法，由长子(firstChild)域指出第 1 个子结点，由兄弟(nextSibling)域将同一个父结点的其它结点串成链表；
6. 树与二叉树的转换：利用长子-兄弟链表表示法，连线-剪枝，即将各兄弟结点连接起来，然后剪断除长子结点外其它兄弟结点与父结点的边，这样形成的二叉树的根结点**没有右子树**，原来的长子结点都在左子树，而它的兄弟结点则成为其右子树上的结点；
7. 森林与二叉树的转换：先将各树转换为二叉树，然后把每棵树的根结点连到上一棵树的右孩子上；
8. 树与森林的遍历：深度优先（先根⇔先序，**后根⇔中序**，可借助长子-兄弟链表法，用二叉树的遍历来实现）；广度优先（层次）；
9. 最小堆 MinHeap：**任一结点**的 key 均≤它的左、右孩子的 key，堆顶（根结点）的 key 是整个堆（树）最小的；即堆是递归定义的，每个堆的子树也是一个堆；注：堆的左、右孩子之间没有确定的大小关系，这与搜索树不同；
   堆是一棵**完全二叉树**，一般采用数组顺序存储，可根据下标直接计算出其父子结点；
   下滑(siftDown)：从某个父结点开始向下比较，如果父结点＞左/右孩子中最小的那个，则交换该父结点和最小的子结点，这样逐层向下调整，直到最下一层（叶结点）；
   上滑(siftUp)：从某个结点开始向上比较，如果该结点＜父结点，则交换该结点和其父结点，这样逐层向上调整，直到堆顶结点；
   建立堆：一种方法是从空堆开始逐个插入结点；另一种是对一般的完全二叉树就地加以调整，即从最后面的一个非叶结点（从 0 开始编号，为 floor((i-1)/2)），即最小的一个子堆）开始，直到根结点，对所有非叶结点从后向前逐个执行 siftDown 调整，自底向上逐步扩大，将各子树局部均调整为堆；时间复杂度均为 O(n*log2 n)；；
   插入结点：新结点总是插入到已形成的**堆的末尾**，然后 siftUp 调整，O(log2 n)；
   删除结点：总是删除**堆顶结点**，然后将**最后一个结点移到堆顶**，最后再执行一次 siftDown 调整；O(log2 n)；
10. 由树的定义可知，从根到达树的每个结点有且只有 1 条路径；扩充二叉树是给所有叶结点赋权的二叉树，非叶结点则没有权值；整棵树的带权路径长度为各叶结点权值与路径长度乘积的和，即$\mathrm{WPL}=\sum_{i=1}^{n_1} w_i \cdot l_i$；
11. Huffman 树是带权路径长度最小的树；Huffman 树不是唯一的；构造算法：
    1) 在全部结点中选择权值最小的 2 个构成一棵树的左、右子结点，其根为这 2 个子结点权之和；
    2) 将上述子树的根作为一个结点，与其它结点一起比较，在这些剩余结点中选择权值最小的 2 个构成一棵树的左、右子结点，其根为这 2 个子结点权之和；可以用**最小堆**帮助找到最小的结点；
    3) 重复上述步骤直到没有结点剩余；
12. Huffman 树没有度为 1 的结点，只有度为 0 和 2 的结点；Huffman 编码，一定是前缀编码；最优判定树；

第6章、集合与字典

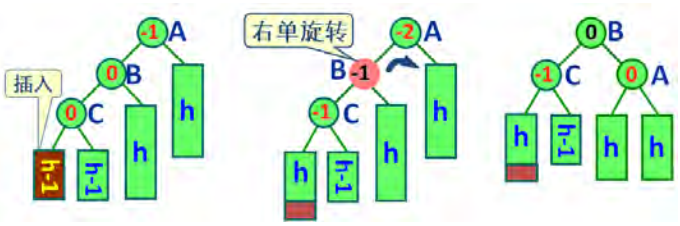
1. 集合中没有重复的元素；
   位向量实现集合：对**有限集合**，可以建立到自然数 0,1,2…n-1 的映射，它的**每个子集**都可以用一个 n 位的**向量来**表示，利用位运算很容易实现集合的相等，并，交，差，补等运算；
   有序链表实现集合：可以表示无穷集合的子集；有序排列便于实现各种运算；
2. 并查集(union-find set 或 disjoint set)，支持 Union，Find 操作；实现并查集的典型方法是采用树（父指针的静态链表方式）来表示元算及其所属子集的关系；合并子集时，将某个子集的根作为另一子集的子结点即可，为提高 Find 的效率，应将小的子集合并到大的子集下面；为进一步提高 Find 效率，可进行压缩路径操作，即将各子结点的父结点都置为根结点；
   等价关系：自反性，对称性，传递性；一个集合 S 中的所有元素可通过等价关系划分为若干互不相交的子集，这些子集称为等价类；利用并查集划分等价类，最初每个元素自成一个等价类，之后每处理一个等价对 i≡j 时，先要确定 i，j 所在集合（Find），如果是不同的集合，则将这 2 个集合合并（Union）；
3. 字典是<key-value>的集合，基本操作是 Find，Insert，Remove；
4. 跳表是有序链表为了方便二分搜索，在首、尾、中间的结点各增加一个指针域的结构；跳表可以有多级；
5. 散列 Hash 是将元素的 key 通过某种较快速的算法直接映射到存储位置，从而直接访问的结构；Hash 必须考虑冲突问题，冲突太多会显著降低效率；
   1) 散列算法要求定义域必须包括全部可能的 key，计算出的地址应均匀分布，算法简单，高效；常用的算法有：
      a.取模：hash(key)=key % p，p 是一个不大于容量 m 的质数；常用；
      b.其它：数字分析法，分析各 key 中数值的分布规律；平方取中法；折叠法：又有移位法，分界法等；
   2) 处理冲突的闭散列方法，发生冲突后占用另外的码位，不能直接删除表中元素，只能标记为已删除（或删除元素后移动后续受影响的全部元素），不适合经常变动的情况；
      a.线性探查法，如果有冲突，则放到下一个空位处，(H+i)%size，即 H+1,H+2,… ；
      b.二次探查法，$(H+i^2)\%size$，$(H-i^2)\%size$，即 H+1,H-1,H+4,H-4,… ；
      c.双散列法，采用两种不同的散列方法；
   3) 处理冲突的开散列方法，用链表（桶）保存同义词，链表内一般采用顺序搜索；
   4) 填充因子=已有记录数/预设最大记录数，表明了发生冲突的可能性，不应过大；

第7章、搜索结构

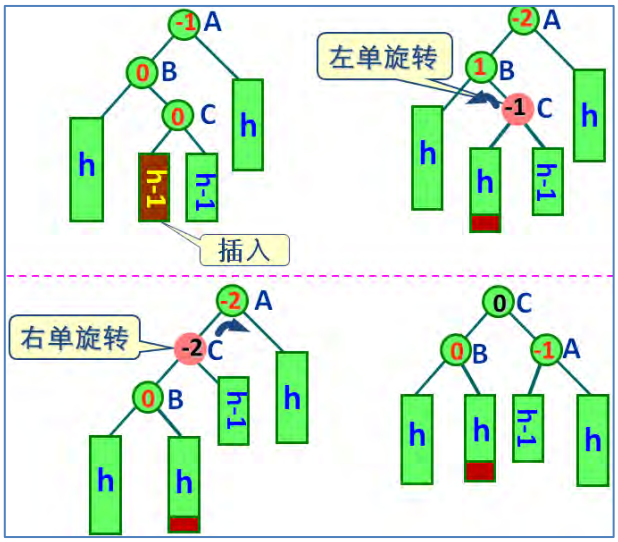
1. 为保持较高的效率，搜索结构在插入和删除等操作后会调整，结构可能发生变化，称为动态搜索结构，反之则称为静态搜索结构；衡量搜索算法效率一般用平均搜索长度 ASL，或磁盘 IO 次数（外部搜索）；
2. 静态搜索结构：1）顺序搜索；2）有序表的折半搜索：mid=low+(high-low)/2，亦即(high+low)/2，但直接相加可能会溢出；判定树，失败结点=叶结点；3）索引顺序表；
3. 二叉搜索树，是构造出的类似折半搜索的判定树那样的树形结构；其特点是左子树（若不为 NULL）上所有结点的 key＜根结点；右子树（若不为 NULL）上所有结点的 key＞根结点；这是递归定义的，每棵子树也都是二叉搜索树；若中序遍历二叉搜索树，则得到的序列是从小到大有序排列的；ASL≤Height；
   1) 二叉搜索树的插入：首先搜索二叉树，检查新插入的元素是否已经在树中，如果已存在，则无需操作；如果不存在，则将其插入搜索停止的结点的左/右子树处；
   2) 二叉搜索树的删除：若为叶结点，直接删除；若仅有左/右子树，则用左/右子树顶替被删除的结点；若既有左子树，又有右子树，则在**右子树中找到 key 最小的结点**，填补到被删除结点位置上，**这是一个递归的过程**；
   3) 若不同的 key 有不同的搜索概率，则 ASL 最小的称为最优二叉搜索树；
4. AVL 树又称高度平衡的二叉搜索树，right.Height - left.Height 的绝对值不超过 1，即只能为-1,0,1，这样可以减少 ASL，提高搜索效率；在插入，删除后要调整结构，以保持平衡；



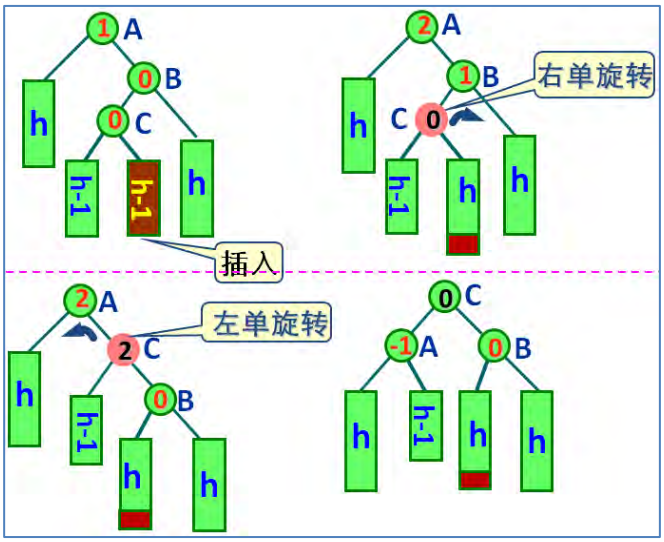
左单旋转



右单旋转



先左后右双旋转



先右后左双旋转

5．伸展树；红黑树（根结点和所有外部结点的颜色都是黑色，从根结点到外部结点的途中没有连续 2 个结点的颜色是红色，所有从根结点到外部结点的路径上都有相同数目的黑色结点）；

第8章、图

1．基本概念

1) 有向图；无向图；不考虑顶点有自环的情况，也不考虑任意 2 个顶点间有多重边的情况；

2) 完全图：任意 2 个顶点间都有边相连，无向图的边数为 n(n-1)/2；有向图为 n(n-1)；

3) **简单路径**，路径上各顶点没有重复；回路/环，路径上首个顶点和最后一个顶点重合；

4) 连通图与连通分量，任意 2 个顶点之间都有**路径**；非连通图的**极大连通子图**叫做连通分量；

5) 强连通图与强连通分量：对**有向图**，每 2 个顶点之间都存在**正反 2 条路径**；**非强连通图**的**极大强连通子图**叫做强连通分量；

6) 生成树(Spanning tree)：对**无向连通图**，是其一个**极小连通子图**，由 n-1 条边构成；

2．图的存储结构，不同的存储结构适合不同的处理算法；

1) 邻接矩阵：大小为 n*n，若边数较少，则矩阵很稀疏，存储利用率低；无向图是对称阵，有向图则不对称；

2) 邻接表：有一个顶点表，按编号顺序保存全部顶点，每个顶点带一个链表，是与此顶点相连的其它顶点在顶点表中的编号，即边的信息；对无向图，每个边保存 2 次，有向图则只有一次；有向图的每个顶点可带一个出边链表，和一个入边链表；边链表的每个结点还可以增加权值域；

3) 邻接多重表：特别适合以边为处理对象，每条边处理一次的算法；这种方法也有一个顶点表，顺序保存全部顶点；但表示边的链表与邻接表不同，每个结点表示一条边，有 5 个域，分别为 Mark，Vertex1，Vertex2，Path1，Path2；Vertex1，Vertex2 是该边的 2 个顶点；Path1，Path2 分别指向 Vertex1，Vertex2 的下一个边结点，这样就分别组成了各自的边链表；没有重复保存边

无向图的顶点表中每个顶点是其边链表的头指针，遍历边时要将当前结点编号与 Vertex1，Vertex2 比较，以判断是选择 Path1 还是 Path2；

有向图的顶点表中每个顶点分别有一个出边链表的头指针和一个入边链表的头指针；

3．图的遍历，为了处理回路，需要标识已经访问过的顶点，

1) 深度优先 DFS，递归算法：访问本结点，然后对**未访问过的**各邻结点 DFS；

2) 广度优先 BFS，非递归，按层访问：出队，访问本结点，将所有邻结点入队，如此循环直到队列为空；

4．最小（代价）生成树 MST (Minimum-cost spanning tree)，一个连通图的生成树必有 n-1 条边，且这些边不能构成环路；带权图的 MST 不是唯一的；下面的算法都属于贪心(Grady)算法

1) Kruskal 算法：**初始时只有顶点，没有边**，然后逐步从边集 E 中取出权值最小的边，如果该边与已有的边不构成回路（准确的说，是该边的 2 个端点不在同一连通分量上），则加入；如此重复，直到加入的边数为 n-1（或边集 E 已空）；O(e*log e)？，适合稀疏图；

用**最小堆**来查找权值最小的边；利用**并查集的查找运算**来判断任一边的 2 个顶点是否来自同一连通分量，用**并查集的合并运算**将新加入的边的顶点各自所在的 2 个连通分量合为 1 个连通分量；

2) Prim 算法：始终将顶点集合分成不重叠的 2 部分，已在 MST 中的顶点集合 V_{mst} 和其它剩余顶点集合 $V-V_{mst}$，称为该图的一个割（cut），2 个集合之间至少有一条边相连，称为桥，从多个桥中选出权值最小的，其 2 个

顶点有一个在 V_{mst}，另一个在 $V-V_{mst}$，将后者移入 V_{mst}，并从边集 E 中删除该桥；如此不断扩大 V_{mst}，直到其中的顶点数为 n，这时里面有 n-1 条边；$O(n^2)$，适合稠密图；

也利用**最小堆**来查找权值最小的边，V_{mst} 每增加一个顶点，需将其另一顶点不在 V_{mst} 中的边插入堆中；

5. 最短路径

1) 非负权值的单源最短路径，Dijkstra 算法：

为了寻找从源点 V_0 到终点 V_t 的最短路径，依次将各中间点 V_p 加入到最短路径点的集合 S 中，计算出源点**经该中间点**到其余所有点 V-S 的最短路径，对集合 V-S 中的点 V_k，如果 V_0 经 V_p 到 V_k 的路径比上一轮得到的最短路径还要短，则更新源点 V_0 到 V_k 的最短路径，记录到与 V_k 对应的辅助单元，并记录下该中间点 V_p 的编号，以作为回溯的线索；distance[k]=min(distance[k], distance[p]+weight(p,k))

若集合 V-S 中的点 V_q 是这一轮中与 V_0 的距离最小的，则将其移入最短路径点的集合 S，实际上每个点只需一个标志域，对已到达的点做标记，就能区分该点是在集合 S 还是在集合 V-S 了；

按上述步骤逐步扩展 S，直到终点 V_t，源点 V_0 到**所有顶点的最短路径都得到了**，自然也就找到了源点 V_0 到终点 V_t 的最短路径；

从终点开始，根据记录的上一个中间点，逐步回溯到源点，逆序输出，就得到完整的路径；

2) 图中任意 2 个顶点之间的最短路径，各边权值均为正数，Floyd 算法：如果用 Dijkstra 算法，可依次以各顶点为源点，每次可计算出该点到其余各点的最短路径，n 次就可得到任意 2 个点之间的最短路径；而 Floyd 算法形式上更直接一些；Floyd 算法原理与 Bellman-Ford 算法基本相似，也是利用了递推公式，图采用邻接矩阵存储，用一个 n*n 的矩阵 A 来保存每次迭代的最短路径，另外用一个 n*n 的矩阵 Path 来记录路径，A 初始化为 A^{-1}[i][j]=weight(i,j)

A^k[i][j]=min(A^{k-1}[i][j], min(A^{k-1}[i][r] + A^{k-1}[r][j]|r=0,1,…n-1))

$Path^k$[i][j]=使上式取得最小值的 r

注意 k 从 0~n-1；

为了确定 i 到 j 的最短路径，若 $Path^{n-1}$[i][j]=s，$Path^{n-1}$[i][s]=t，$Path^{n-1}$[i][t]=u，…，$Path^{n-1}$[i][v]=i，则路径为 i→v→…→u→t→s→j；

6. 用顶点表示活动的网络 AOV (Action on Vertex)：AOV 是一个有向无环图 DAG (Directed Acyclic Graph)，用来表示项目的流程；AOV 中各顶点存在着前驱/后继关系，拓扑排序可将各顶点排列成一个线性的有序序列，使得 AOV 中存在的所有前驱/后继关系都得到满足，但处理过程中会导致人为增加一些前驱/后继关系；若通过拓扑排序能将 AOV 的全部顶点都排入一个序列中，说明该 AOV 网络中没有有向环；否则存在有向环，说明该流程不可行；拓扑排序的算法为

1) 计算 AOV 各顶点的入度（直接前驱的个数）；
2) 选择入度为 0（没有直接前驱）的一个顶点输出；
3) 从 AOV 中删除该顶点和它发出的所有有向边（该点入度为 0，故没有指向该顶点的有向边），即将该顶点指向的所有顶点的入度减 1；这时可能产生多个入度为 0 的顶点，为避免后续反复搜索顶点表中入度为 0 的顶点，可将它们压入一个栈保存起来（使用一个可变长的数组或队列保存这些顶点都是可行的）；
4) 从栈中弹出一个入度为 0 的顶点，重复上步，删除该顶点和它发出的所有边，保存新产生的入度为 0 的顶点，直到没有入度为 0 的点为止（栈为空）；如果这时 AOV 中还有顶点，说明该 AOV 存在有向环；
5) 一个 AOV 的拓扑有序序列不是唯一的；

拓扑**逆序**是在上述算法中每次删除**出度**为 0 的顶点及**指向**该顶点的有向边；

7. 用边表示活动的网络 AOE (Action on Edge)：同样用来表示项目的流程，但活动用有向边来表示，边的权值代表该活动持续的时间，顶点则表示活动的开始或结束事件；AOE 也不存在有向环，且只有一个入度为 0 的顶点，称为源点，记为 V_0，即项目的开始事件，也只有一个出度为 0 的顶点，称为汇点，记为 V_{n-1}，是项目的结束事件；

从源点到汇点的所有路径中，**最大长度**的路径称为关键路径 Critical Path，这是完成整个项目所需的**最短时间**；

记 VEarly[i]为顶点 V_i 的最早可能开始时间，须前面活动都完成了，V_i 才能开始；即源点 V_0 到 V_i 的关键路径长度；

记 Vlast[i] 为顶点 V_i 的最迟允许开始时间，须保证项目按时完工，是以 V_i 为源点，到汇点 V_{n-1} 的关键路径长度；

活动 A_i 是以顶点 V_i 为起点，到顶点 V_j 的有向边，duration(i,j)是 A_i 持续的时间，即所需的工期；

记 Early[i]为活动 A_i 的最早可能开始时间，可知 Early[i]=VEarly[i]；

记 Last[i] 为活动 A_i 的最迟允许开始时间，可知 Last [i]=VLast[j] - duration(i,j)；

Last[i]-Early[i]称为活动 A_i 的**松弛时间**，是最早开始时间与最迟允许开始时间的裕量；

在关键路径上的活动，其松弛时间必为 0，即 Last[i]=Early[i]；计算出所有活动的松弛时间，为 0 的即为关键路径上的活动；

VEarly[i]的递推公式为：

VEarly[0] = 0；

VEarly[i] = max(VEarly[k] + duration(k,i)|k)，V_k 是 V_i 的所有直接前驱，即要求各顶点是拓扑排序的；从源点向终点依次递推，最终到达终点，VEarly[n-1]，就是项目的总工期；

VLast[i]的递推公式为：

VLast[n-1] = VEarly[n-1]，这是上一步得到的项目的总工期；

VLast[i] = min(VLast[j] - duration (j,i)|j)，V_j 是 V_i 的所有直接后继，即要求各顶点是拓扑逆序的；从终点向源点反向递推；

再由 VEarly[i]，VLast[i]和 duration (i,j)，计算出各活动的 Early[i]和 Last[i]，并计算出松弛时间，就找出了关键路径上的活动；

第9章、排序

1. 排序算法的稳定性；内部排序与外部排序；性能评估：数据比较次数和数据移动次数；
2. 插入排序，对基本有序的序列比较高效
   1) 直接插入排序：当插入第 i 个元素时，前面的 0~i-1 个元素已经有序，只要将 i 与这些元素从后向前依次比较，即可找到正确的位置，并将后面的元素依次移动；
   2) 折半插入排序：又称二分法插入排序，在寻找 i 的插入位置时采用折半搜索；
   3) 希尔 shell 排序：又称缩小增量排序；先取一个 gap，将全部元素分为 gap 个序列，所有距离为 gap 的元素放在一个子序列里，在每个子序列进行直接插入排序；然后缩小 gap 重复上述子序列划分和排序，直到 gap=1；各 gap 必须是互质的，可取 gap=floor(gap/3)+1；排序开始阶段 gap 较大，每个子序列元素较少，排序速度较快，到后期，大多数元素已基本有序，也使排序速度较快；
3. 交换排序
   1) 冒泡排序：从 n-1 号元素开始，相邻的两个元素相比较，如果逆序则交换，使最小的元素到达 0 号位置，然后对 1~n-1 号元素组成的子序列继续上述操作；
      改进方法：若一轮比较后没有元素交换，说明序列已经有序，则结束排序，需一 bool 量记录是否发生了交换；
   2) 快速排序：是一种分治算法；任取某个元素作为基准，将整个元素序列分为左右 2 个子序列，左侧子序列中所有元素都＜该基准，右侧子序列中所有元素都＞该基准，基准元素的位置即**最终**排序后的位置；然后分别对左，右 2 个子序列递归执行上述操作，直到子序列长度为 1；

   快速排序的最坏情况是序列本来就有序（包括正序或逆序）的情况，当元素个数较少时，往往比简单排序方法还要慢；因此有改进算法先执行快速排序几个迭代，得到基本有序的序列，再执行直接插入排序，或当划分后的序列较小时改用插入排序；

   基准元素的选择也会影响快速排序的效率，一种办法是随机选择，另一种是取首、尾、中部三个值的中间值；

   三路划分的快速排序是处理**有较多相等元素情况**的算法；
4. 选择排序：适合选取 top i 这样的需求；
   1) 直接选择排序：将一组元素中的最小值放置在首位，然后对剩余的元素重复上步，直到剩余 1 个元素为止；
   2) 锦标赛排序：利用胜者树/败者树，是稳定的；时间复杂度为 O(n*log2 n)，需要 n-1 个辅助空间；
   3) 堆排序：将一维数组视为顺序存储的完全二叉树，逐步 siftDown，将其调整为最大堆，然后将堆顶（0 号，最大值）与最后一个（n-1 号）元素交换，将 0~n-2 号元素视为堆，siftDown 调整后，再将新的堆顶与 n-2 号交换，如此重复，直到堆内只有 1 个元素为止；
5. 归并排序：将待排序的 n 个元素分成前、后 2 个长度相等的子序列，为每个子序列排序，然后再将它们合并为一个序列，这称为两路归并排序，可递归实现；归并两个长度分别为 m，n 的有序序列最坏的情况需比较 m+n-1 次；
6. 上述的排序算法都是基于数组实现的；用链表则无需移动元素，亦可用静态链表，排序完成后再顺序输出；
7. 分配排序：非比较算法；多个 key 的排序；分配，元素从桶上面依次进入；收集，元素从桶下面依次移出；
   1) 桶式排序
   2) 基数排序：高位优先 MSD；低位优先 LSD，从最低位到最高位，每位作为 key 做一轮分配和收集（排序），**要求每轮排序必须使用稳定的算法**；

交换类排序和**选择**类排序在每轮排序后，已处理过的元素所在的位置即是最终有序状态下的位置；而**插入**排序和**归并**排序没有这个特点；

注 1：快速排序在序列已经有序（正序或逆序）时为最坏情况，而其它方法一般是正序为最好情况，逆序为最坏情况；

注 2：基数排序中 d 指数字的位数，r 是每位数字的范围；这虽然是线性的复杂度，但辅助操作，如截取各位的 key

消耗的时间是比较大的，所以大多数情况不一定比快速排序效率高；

排序算法		时间复杂度			空间复杂度	稳定	备注
		平均	最好	最坏			
插入	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	是	适合小规模或基本有序
	shell 排序	$\sim O(n^{1.25})$		$O(n^2)$	$O(1)$	否	适合中等规模
交换	改进的冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	是	
	快速排序	$O(n*\log_2 n)$	$O(n*\log_2 n)$	$O(n^2)$注 1	$O(\log_2 n)$	否	平均情况下性能最好
选择	直接选择	$O(n^2)$			$O(1)$	否	
	堆排序	$O(n*\log_2 n)$			$O(1)$	否	辅助空间少
归并排序		$O(n*\log_2 n)$			$O(n)$	是	稳定，但辅助空间大
基数排序		$O(d*(n+r))$注 2			$O(r)$	是	适合大规模，key 少

第10章、文件、外部排序与搜索

1. 稠密索引，稀疏索引；
2. 倒排文件；次索引的二进制位矩阵法；
3. 外排序需要多次从外存与内存间交换数据，决定效率的关键在于减少外存的 IO 次数；多使用归并排序，增大归并路数 k，可减少 IO 次数，但路数过多，每次交换的数据量小，又会增加 IO 次数，k 的选取与内存的缓冲区大小和磁盘性能都有关，需综合考虑；k 路归并对应正则 k 叉树，即只有度为 0（叶结点）和 k 的结点的树；
4. 归并排序分两个阶段
   1) 将文件中的数据分段输入到内存，然后用某种内部排序方法对其排序，得到初始归并段，然后将其写回外存；
   2) 对多个初始归并段多遍归并，最终形成单一的归并段，就完成了外排序；
5. 初始归并段的生成，采用置换-选择排序方法：当工作区满后（初始从输入缓冲区复制数据），将工作区调整为败者树，选择最小的元素输出到归并段（输出缓冲区），然后从输入缓冲区获取新的元素，如果新元素比当前归并段的最后一个元素（亦即最大者，可看做动态的门槛值）要小，则不能作为当前归并段的一部分，只能等待下一个归并段，将其放至工作区首（或尾，只要是固定位置顺序放置即可），当前工作区（名义）容量减少一个单元，否则就将其放入插入工作区已形成的败者树中，选出比当前归并段门槛大的最小值，插入归并段尾；重复上述操作，直到所有新输入的元素都不能加入当前归并段，即工作区（名义）容量为 0，完成当前归并段，恢复工作区（名义）容量，开始生成下一归并段，直到全部数据都处理完；
   常规方法每次的归并段大小等于工作区容量，而采用置换-选择排序方法生成的归并段都大于或等于工作区容量，最好的情况是输入的元素是有序的，则只生成一个归并段，最坏的情况是输入的元素是逆序的，生成的归并段大小等于工作区容量，平均情况的归并段长度不同；
6. 置换-选择方法生成的归并段长度不同，在第 2 阶段归并时，让元素少的段先归并，元素多的段后归并，于是有最佳归并树的概念，即带权路径长度 WPL 最小的正则 k 叉树；k 路最佳归并树可参考二叉树的 Huffman 算法，初始状态，叶结点是各归并段的长度，每次合并 k 个最小结点，并将其和作为根结点的值，作为一个新结点，继续参与合并；每个叶结点到根结点的路径长度是该结点参与归并的次数，每次归并有 2 次 IO（I 算一次，O 算一次），若每个归并段的 n 个元素分别读写，则一个归并段本身需要 n 次 IO，完成全部归并需要的 IO 次数=2*WPL；
7. 败者树也是一棵完全二叉树，采用一维数组存储，必须从 1 号下标开始存储（0 号单元用来保存整棵树的胜者）；
   归并用的败者树的内部结点和叶结点的**类型不同**，叶结点是数据元素，在 k 路归并算法中对应 k 个归并段的队首，仍存储在各归并段，败者树实际只有内部结点，其值为对应归并段的编号；
   败者树没有度为 1 的结点，只有度为 2 的内部结点和度为 0 的叶结点；由二叉树的性质，$n_2=n_0-1$；
   k 路归并的败者树有 k 个叶结点，编号从 k~(2k-1)（对应 0~k-1 号归并段，归并段从 0 开始编号），有 k-1 个内部结点，编号从 1~(k-1)（加上 0 号单元，败者树的大小恰好为 k）；
   第 i 号叶结点（对应 i-k 号归并段）的父结点为 floor(i/2)号；
   父结点总是保存 2 个子结点中的败者对应的归并段编号，若从小到大归并，则小的元素是胜者，大的是败者；
   建立树时，从最后一个叶结点开始，2 个子结点组成一个小组进行比较，将败者的归并段编号保存到它们的父结点，胜者的归并段编号保存到父结点对应的辅助单元；这样对应每个父结点都需要一个辅助单元保存胜者的编号，即一个大小为 k-1 的辅助数组；
   由各结点的编号 i，可知此结点在第 ceil(log2(i+1))层（根结点为第 1 层），决定了此结点参与比赛的轮次；第 1 轮比赛都在叶结点之间进行，但有部分叶结点直接晋级下一轮比赛，第 2 轮有可能是叶结点之间的比赛，也可能是叶结点和内部结点的比赛，或内部结点之间的比赛；

下一轮仍从后向前，即本层最右边结点开始进行比较，但有内部结点参与的比赛规则有变化：若右兄弟若为叶结点，左兄弟结点为内部结点，则叶结点与胜者，即左兄弟对应的辅助单元比较，将败者的归并段编号保存到父结点，胜者编号保存到对应的辅助单元；若左、右兄弟均为内部结点，则分别将其对应的胜者，即分别对应的辅助单元相比较，败者编号存入父结点，胜者编号存入对应的辅助单元；

重复上述步骤，直到第 3 号结点与第 2 号结点比较，败者编号保存到其父结点（1 号），即根结点，将胜者编号保存到败者树存储数组的 0 号结点（辅助单元可用可不用），这是 k 个叶结点中最小元素的归并段编号，至此败者树建立完成，比较的次数为内部结点个数，即 k-1；

k 路归并时，将败者树的胜者，即 0 号结点对应的归并段队首放到生成的归并段尾，败者树空缺的位置由原归并段下一个元素替补，这时需要调整败者树，方法是将新元素与父结点保存的败者编号对应元素进行比较（这里需要一个辅助单元保存刚才取走的元素），将败者的编号保存到父结点，胜者继续与上一级父结点保存的败者比较，直到根结点；

上面建立败者树的方法比较复杂，利用败者树的调整方法和恰当的初始化可以简化败者树的建立过程：初始化败者树各内部结点和 0 号结点都为 k（这就好像存在一个虚拟的编号为 k 的归并段，其中只有一个 MinValue 元素，它打败了 0~k-1 号的全部归并段，即这时已符合败者树的要求），然后从最后的 k-1 号归并段（2k-1 号叶结点）向前逐个参加比赛，调整败者树，直到 0 号归并段（k 号叶结点）为止；这种方法可以省去各内部结点对应的辅助单元；在归并过程中，若某个归并段的元素都被取走，则将其对应的叶结点设置为最大值 MaxValue，直到所有叶结点都为 MaxValue，全部归并段就归并完成了；

8. B 树：一棵 m 阶（m>2，即最少为 3 叉）的 B 树是一棵平衡的 m 路搜索树，可为空树；一棵非空 m 阶 B 树的性质：
   1) 根结点至少有 2 个子树；
   2) 其它内部结点至少有 ceil(m/2)个子树，最多有 m 个子树；注，若 m 为 unsigned int，则 ceil(m/2)=(m+1)/2；
   3) 内部结点中的 key 比子树数少 1 个，key[0]不使用，或用来保存子树个数 n（这样 key 数组实际有 n+1 个元素，而有的实现中则为 n 个元素），key[1]~key[n]保存实际的值，且正序排列；该结点的 0 号**子树**的 key 都＜key[1]，n 号**子树**（最后一棵）的 key 都＞key[n]，而 i 号（0<i<n）**子树**的 key 在 key[i]和 key[i+1]之间；
   4) 各结点也可以是包含 key 和地址的索引项；
   5) B 树类似于稀疏的多级索引，所有的 key 分布在各内部结点，搜索过程是在一个结点内顺序或折半查找，如果找到了就直接返回，否则亦能判断范围，选中某棵**子树**继续向下一层查找；直到最底层为 NULL 指针，说明查找失败；
   6) 所有叶结点都在同一层，它们没有子树，指向子树的都是 NULL，这些实际不存在的 NULL 子结点称为失败结点；
9. B 树是从空树逐个插入 key 而建立的；B 树插入 key 的过程与二叉搜索树类似，首先要从根开始搜索，如果找到则无需插入，否则搜索停止的位置即新 key 要插入的位置，由此可知，插入操作总是发生在叶结点上，需判断插入结点内 key 的个数是否满足 B 树的规定（(m+1)/2-1~m-1 个 key），如果超出了，则要将该结点拆分，方法是将结点中间的某个 key（一般取 key[(m+1)/2]）上移一层到父结点，必定是插入父结点的 key[n+1]处，同时当前结点被分成两部分，父结点的 key[n]对应左侧部分，新加的结点 key[n+1]对应右侧部分；向父结点增加 key 可能导致连锁反应，最坏的情况是整棵 B 树增加一层；
10. 从 B 树删除指定 key 后也要考虑不能使对应结点内 key 的个数＜(m+1)/2-1，否则需处理；

    若该结点是叶结点，
    1) 且该结点的左、右兄弟结点中某个结点的 key 的个数＞(m+1)/2-1 的，则从该兄弟结点借 1 个 key 到当前结点，为保证各结点的 key 之间的大小关系，借来的 key 不能直接替换删除留下的空位，应将当前结点的父结点对应的 key 放到删除留下的空位处，从兄弟结点借来的 key 放到父结点腾出的空位处；
    2) 且该结点所有的左、右兄弟结点的 key 的个数都是(m+1)/2-1，则在当前结点删除 key 后，可将剩余(m+1)/2-2 个 key 与左兄弟或右兄弟合并，同时还要从父结点的 key 中取下 1 个来参与合并，若与左兄弟（i-1 号子树）合并，则取父结点的 key[i-1]；若与右兄弟（i+1 号子树）合并，则取父结点的 key[i+1]；从父结点删除 1 个 key 后也可能导致连锁反应，最坏的情况是整棵 B 树减少一层；

    若该结点不是叶结点，需将其与相邻 key 逐层交换，直到叶结点，然后再用上述方法删除；相邻 key 是指要删除的 key 对应子树的最小 key，或邻近的左子树的最大 key，交换后不会破坏 B 树的性质，且能使该结点下移一层；
11. B+树是 B 树的变形，特点是：
    1) Key（也可为含 key 和地址的索引项）都在叶结点层，从小到大顺序排列，且各叶结点组成一个有序链表；
    2) 内部结点只是保存对应子树中最大（或最小）的 key 本身，不会保存索引地址等内容，比 B 树的结点小很多；
    3) 内部结点内 key 的个数和子树的个数相同；
    4) 与 B 树一样，对结点子树的个数有限制，根结点至少 2 棵子树，其它内部结点至少有 ceil(m/2)个子树，各

内部结点最多有 m 个子树；

5) B+树除了一个指向根结点的指针外，还有一个指向 key 最小的叶结点的指针，亦即叶结点组成的链表的头指针；对 B+树的搜索可以像 B 树那样从根结点开始，先在结点内查找，如果没有找到，可由结点内 key 的信息判断出在那棵子树，然后沿该子树继续搜索；但由于 B+树内部结点只有 key，所以不论是否在内部结点找到该 key，都要继续搜索到叶结点，在叶结点中查找到地址等信息；
此外还可以直接从子结点链表开始顺序搜索，方便顺序遍历子结点；

6) B+树插入 key 和删除 key 与 B 树相似，也需要判断结点内 key 的数量是否满足规定，否则需分裂/借入或合并，不同之处是 B+的操作都在叶结点上进行；

12. 可扩充散列，二叉 Trie 树，目录表；Trie 树，适合可变长 key（如文件名）的多级索引；

```
int partition(int a[],int low, int high) {
    //单向
    int piovtPos=low;
    int piovt=a[piovtPos];
    for (int i = low+1; i <= high; i++) {
        if (a[i]<piovt) {
            piovtPos++;
            if (piovtPos!=i) {
                swap(a[piovtPos],a[i]);
    }   }   }
    swap(a[low],a[piovtPos]);
    return piovtPos;
//  另一种方法，前后交替
//  int piovt=a[low];
//  while(low<high) {
//      while(low<high&&a[high]>=piovt) {
//          high--;
//      }
//      a[low]=a[high];
//      while(low<high&&a[low]<piovt) {
//          low++;
//      }
//      a[high]=a[low];
//  }
//  a[low]=piovt;
//  return low;
}

void quickSort(int a[],int low, int high) {
    if (high>low) {
        int piovtPos=0;
        piovtPos= partition(a,low,high);

        quickSort(a,0,piovtPos-1);
        quickSort(a,piovtPos+1,high);
    }
}

void bubbleSort(int a[],int len) {
    bool hasSwap=true;
    for (int i=0; i<len && hasSwap; i++) {
        hasSwap=false;
        for (int j = len-1; j>i; j--) {
            if (a[j]<a[j-1]) {//若相邻两元素逆序
                swap(a[j],a[j-1]);
                hasSwap=true;
            }
        }
    }
}

void insertSort(int a[],int len) {
    int temp,j;
    for (int i = 1; i < len; i++) {
        temp=a[i];
        j = i-1;
        for (; j>=0&&temp<a[j]; j--) {
            a[j+1]=a[j];
        }
        a[j+1]=temp;
    }
}

void shellSort(int a[],int low,int high) {
    int temp,i,j,gap = high-low+1;
    do {
        gap=gap/3+1;
        for (i = low+gap; i <=high; i++) {
            temp=a[i];
            j = i-gap;
            //注意&&条件顺序
            for (; j>=low&&temp<a[j]; j-=gap) {
                a[j+gap]=a[j];
            }
            a[j+gap]=temp;
        }
    } while (gap > 1);
    //确保 gap=1 执行且只执行 1 次；
}

void selectSort(int a[],int len) {
    int k;
    for (int i = 0; i < len; i++) {
        k=i;
        for (int j = i+1; j < len; j++) {
            if (a[k]>a[j]) {
                k=j;
            }
        }
        if(k!=i) {
            swap(a[i],a[k]);
        }
    }
}

void heapSort(int a[],int len) {
    initHeap(a,len);
    for (int i=len-1; i>=0; i--) {
        a[i]=removeHeapTop(a,i+1);
    }
}//计算堆的父子节点，要从 0 开始编号
```

```
void merge(int src[],int low,int mid,
           int high,int dst[]) {
    for (int k = low; k <= high; k++) {
        dst[k]=src[k];
    }
    int t=low, i=low, j=mid+1;
    while (i<=mid && j<=high) {
        if (dst[i]<dst[j]) {
            src[t++]=dst[i++];
        } else {
            src[t++]=dst[j++];
        }
    }
    while(i<=mid) {
        src[t++]=dst[i++];
    }
    while(j<=high) {
        src[t++]=dst[j++];
    }
}

void mergeSort(int a[],int low,int high,
               int aa[]) {//aa[]为辅助空间
    if (high>low) {
        int mid=(high-low)/2+low;
        mergeSort(a,low,mid,aa);
        mergeSort(a,mid+1,high,aa);
        merge(a,low,mid,high,aa);
}   }

void mergeSort(int a[],int len){
    int* aa=new int[len];
    mergeSort(a,0,len-1,aa);
    delete[] aa;
}

//大顶堆相关函数
void siftUp(int a[],int start) {
    int parent=(start-1)/2;
    while (parent>=0&&a[parent]<a[start]) {
        swap(a[parent],a[start]);
        start=parent;
        parent=(start-1)/2;
    }
}

void insert(int a[],int len,int value) {
    a[len]=value;
    siftUp(a,len);
}//需调用方保证不会溢出
```

```
void siftDown(int a[],int len,int start) {
    int left,right,max;
    for(left=start*2+1;left<len;) {
        max=left;
        right=left+1;
        if (right<len&&a[left]<a[right]) {
            max=right;
        }
        if (a[start]<a[max]) {
            swap(a[start],a[max]);
            start=max;
            left=start*2+1;
        } else {
            break; //要求子树已经调整为堆;
        }
    }
}

int removeHeapTop(int a[],int len) {
    int top=a[0];
    a[0]=a[len-1];
    siftDown(a,len-1,0);
    return top;
}

void initHeap(int a[],int len) {
    for (int i=len/2-1; i>=0; i--) {
        siftDown(a,len,i);
    }
}
```

计算机组成原理

《计算机组成与系统结构》袁春风 清华大学出版社 ISBN 978-7-302-21905-7

第1章、计算机系统概述

1．计算机硬件的发展：第一代—电子管；第二代—晶体管；第三代—中小规模集成电路；第四代—超大规模 IC；
2．冯·诺依曼体系结构：控制器，运算器，存储器和输入/输出设备 5 部分；指令和数据用二进制表示，两者均存放在存储器中，按地址访问；指令由操作码和地址码组成；采用“存储程序”的方式工作；传统的计算机的组织结构以运算器为中心，现在发展为以存储器为中心；
3．软件和硬件的分界面是指令集体系结构 ISA；
4．计算机的层次结构：~~应用程序~~，高级语言，汇编语言，操作系统，机器语言（，微指令语言）；
5．基本性能指标：程序的响应时间，系统的吞吐率；CPU 的基本性能参数：时钟周期（CPU 中最小的时间单位）、主频（时钟周期的倒数）、CPI（执行指令所需的**时钟周期**，与特定指令相关；对 CPU 或程序，取加权平均值）；CPU 执行时间是时钟周期，CPI 和指令条数三者的乘积；MIPS（定点指令数）、MFLOPS（浮点运算次数）；基准程序；

类型	说明
时钟周期/节拍	CPU 中最小的时间单位，主频的倒数
机器周期/CPU 周期	执行指令的基本操作（取指，译码等）所需时间，含多个时钟周期，现代已不多用此概念
微指令周期	读出并执行一条微指令的时间，一般即机器周期
指令周期	取出并执行一条指令所需时间，含多个时钟周期
存取时间	又称存储器访问时间，完成一次读/写的时间
存取周期/存储周期	进行连续两次独立的存储器操作所需的最小间隔，一定大于存取时间
总线周期	CPU 通过总线对主存或 IO 端口进行一次访问的时间

第2章、数据的机器级表示

1．从 ISA 的层次来看，计算机底层的机器级只有几类简单的基本数据类型，由它们可以组合成各种复杂类型的数据；
2．二进制→十进制：若有 n 位整数，m 位小数，按权展开，为$\sum_{i=1}^{n} b_i \cdot 2^{i-1} + \sum_{i=1}^{m} b_i \cdot 2^{-i}$；
十进制整数→二进制：除 2 取余，倒序排列；十进制小数→二进制：乘 2 取整（舍掉整数），顺序排列，最后在前面加上 0.□□（可能需要近似）；若既有整数，也有小数，整数和小数分别转换后合并即可；
3．定点数 — **原码**：二进制（绝对值）的真值，在最高位加 1 位符号位；有+0 和-0；加减运算需区分符号位；定点原码小数用来表示浮点数的尾数部分；后面提到的真值也都是指绝对值；
反码：正数的反码与原码相同，负数的反码是真值按位取反；同样有+0 和-0；很少使用；
补码：定点**整数**和**小数**的正数的补码均与原码相同；**对**负数，**定点整数**的补码是反码+1，即真值按位取反后（末位）+1（带进位）；0 的编码是唯一的，负数的个数比正数多 1 个，即 n 位补码（1 位符号+（n-1）位**数值**）的范围为-2^{n-1}~0~$(2^{n-1}-1)$；如-1， n 位补码为$2^n - 1\ (\%2^n) = 11\ldots1$（n 个 1）；**定点小数**的补码是即真值按位取反，**末位**+1（带进位），表示的范围为$-1 \le X < 1$；-1 作为小数的 n 位补码为$2 - 1 = 1.00\ldots0$（n-1 个 0）；加减运算不必区分符号位，与数值位一同参与运算；实质是模2^n运算；整数和小数**不能合并**求补码，需单独处理；
移码：只适用于表示整数，n 位真值加上偏置常数2^{n-1}，使所有值都为**非负整数**，主要用于表示浮点数的阶；一个数的移码是其补码最高位取反，其余位不变；
4．同样的位数，浮点数的个数并不比定点数多，只是范围变大了，在数轴上不等距分布；以单精度（32bit）为例，IEEE 754 浮点数格式如下（注意阶码的偏置不是 128，而是 127，扩大了表示范围）：

类型	符号 1bit	阶码 8bits	尾数 23bits	二进制数值
正零/负零	0 / 1	00000000（ 0）	全 0	$+\mathbf{0} / -\mathbf{0}$
正无穷/负无穷	0 / 1	11111111（255）	全 0	$+\infty / -\infty$
无定义数	0 / 1	11111111（255）	$\neq$ 0（共$2^{23}-1$个）	NaN
规格化非零数	0 / 1	$0 < e < 255$	$f = 00\ldots0 \sim 11\ldots1$	$1.f \times 2^{e-127}$
非规格化数	0 / 1	00000000（ 0）	$f \neq 0$	$0.f \times 2^{-126}$

5．十进制的 BCD 码（8421 码等），逻辑值，西文和中文字符与字符串；大端（Big Endian）/小端（Little Endian）

6．常用数据校验码：奇偶校验码，海明校验码，循环冗余校验码（CRC）；检错，纠错；

7．**奇/偶校验**：若数据为$B=b_{n-1}b_{n-2}\ldots b_0$，当有奇数/偶数个 1，取检验为$P=0/1$，即使得$PB$中 1 的总个数为奇数/偶数，用位运算表示，奇校验时$P=b_{n-1}\oplus b_{n-2}\oplus\ldots\oplus b_0\oplus 1$，偶校验$P=b_{n-1}\oplus b_{n-2}\oplus\ldots\oplus b_0$；
能检出奇数个 bit 错误的情况（不论奇校验还是偶校验，两者效果一样），不能发现偶数个 bit 出错，实现简单，常用于字节长度的数据检错；

8．⊕为异或运算，亦记为 XOR，$A\oplus B=\overline{A}B+A\overline{B}$，C 语言表示为 a^b == (~a)&b|a&(~b)；异或满足结合律；
若 *A*，*B* 相同，结果为 0，不同则为 1；若$C=A\oplus B$，则$\overline{A}=B\oplus C$；若$C=0$，说明 *A*，*B* 相同，即$A=B$；
多个 bit 的异或结果，当奇数个 1 为 1，偶数个 1 为 0，故用于生成奇偶校验码；
$A\oplus A=0$，$A\oplus 0=A$，由此得到不用临时变量交换两个数的代码 a=a^b; b=a^b; a=a^b;

9．**海明校验**：实质是多重（分组）奇偶校验；
若有 *k* 位校验位，故障字也为 *k* 位，则可表示2^k个状态，若有 *n* 位数据，与校验码一起存储，对于只发生 1 位出错的情况，有$n+k$种，加上 1 个无错的状态，则要求$2^k\geq n+k+1$，由此可计算出 *k* 的值；
由$n+k$位数值进行位运算，得到 *k* 位故障位，规定故障字的含义如下：
当故障字全 0 — 表示正确状态；
当故障字只有 1 位为 1 — 表示该校验位出错；校验位出错无需纠正；
当故障字有多位为 1 — 表示相应的无符号整数（十进制）是出错位，将该位翻转即可纠正；
由此可知，*k* 位校验位应放在2^{i-1}（$i=1\ldots k$）位处，*n* 位数据则顺序填充余下的位置；
例如：6 位数据，计算校验位数 *k*，由$2^k\geq 7+k$，显然$k>3$，而$k=4$刚好满足不等式，故应有 4 位校验位，分别位于数据的第 1，2，4，8 位，故障字也为 4 位，数据位 + 校验位共 10 位；
故障字 0100 表示第 4 位，亦即第 3 个校验位出错，可以忽略；这说明第 4 位码与第 3 位**故障位**有关；
故障字 0110 表示第 6 位，亦即第 3 个数据位出错，将其翻转就可纠正；这说明第 6 位码与第 3，4 位**故障位都**有关；
故障位和各码位的关系如下表：

	M_{10}	M_9	M_8	M_7	M_6	M_5	M_4	M_3	M_2	M_1
	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1
e_4	√	√	√							
e_3				√	√	√	√			
e_2	√			√	√			√	√	
e_1		√		√		√		√		√
故障字$e_4e_3e_2e_1$	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001

故障位的计算方法为：
$e_4=M_{10}\oplus M_9\oplus M_8 = D_6\oplus D_5\oplus P_4$
$e_3=M_7\oplus M_6\oplus M_5\oplus M_4 = D_4\oplus D_3\oplus D_2\oplus P_3$
$e_2=M_{10}\oplus M_7\oplus M_6\oplus M_3\oplus M_2 = D_6\oplus D_4\oplus D_3\oplus D_1\oplus P_2$
$e_1=M_9\oplus M_7\oplus M_5\oplus M_3\oplus M_1 = D_5\oplus D_4\oplus D_2\oplus D_1\oplus P_1$
正确状态下，$e_4e_3e_2e_1=0000$，由异或运算性质（若$A\oplus B=0$，则 *A*，*B* 相同，即$A=B$），可得
$P_4=D_6\oplus D_5$　$P_3=D_4\oplus D_3\oplus D_2$　$P_2=D_6\oplus D_4\oplus D_3\oplus D_1$　$P_1=D_5\oplus D_4\oplus D_2\oplus D_1$
校验时，直接用各码元计算故障字，如$e_4=M'_{10}\oplus M'_9\oplus M'_8$，即可知是否有错，及错误码位；
或由 D'_i 计算出 P''_j，则 $e_j=P'_j\oplus P''_j$；
海明距离指两个码字的对应比特取值不同的位数，简称码距；在一个有效编码集中，任意两个码字的海明距离的最小值称为该编码集的海明距离；
海明码如果要**检测** d 位错误，需要码距 d+1 的编码方案；如果要**纠正** d 位错误，则需要码距 **2d+1** 的编码方案；
上面的例子中，每个数据位至少参与了两个校验码的生成过程，当该位翻转时，会导致码字中的 3 位发生变化（2 校验位+该位），故码距为 3，可纠正 1 位错误，这正是我们的要求，还可知该编码方案能检测出 2 位出错的情况，但不能同时发现两位错和纠正一位错，这种方案称为 SEC (Single Error Check)；能同时发现两位错和纠正一位错的编码方案称为 SEC-DED (Double Errors Detection)，需增加一位校验位；还可知**奇偶校验的码距为 2**；

10．**循环冗余校验码（CRC）**：若 M 为 n 位二进制数据，CRC 校验码有 k 位；将 M 左移 k 位，除以（模 2 运算除法）一个约定的 k+1 位常数 G（生成多项式），会得到 k 位余数，即 M 关于 G 的 CRC 校验码，将其拼接到 M 后，即$D=M\times 2^k+R$，一起发送或存储；注，k+1 位除数得到的余数一定是 k 位，计算除法前将 M 左移 k 位仅为了给 k 位余数 R 留出位置；

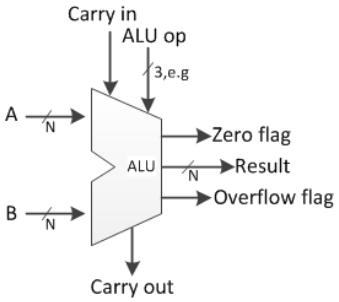
检错时，将 D 也除以 G，若余数为 0，则说明没有错误，否则有错；不同的余数对应不同的出错位，可据此**纠错**；
多项式$M(x)$与二进制数$M = m_n m_{n-1} \ldots m_1$的对应关系为$M(x) = \sum_{i=1}^{n} m_i \cdot x^{i-1}$，如 1001 对应$x^3 + 1$；
模 2 运算除法的加减法均为异或（XOR）运算，相同为 0，不同为 1，**不进位**；部分余数首位为 1 则商 1，否则商 0；
当部分余数比除数位数少 1 位时停止计算，另外，如果将余数补 0，继续除下去，那么一组余数会循环出现，这就是 CRC 的由来；事实上，若$(M \times 2^k) \div G = Q \ldots R$，则$(M \times 2^k) = G \times Q + R$，
$D = M \times 2^k + R = G \times Q + R + R = G \times Q$（R+R 为异或运算），即余数为 0；
CRC 是面向 bit 流的，适用于磁盘和网络传输等场合；网络应用时，用于校验较长的一帧，并且只用于检错，不用于纠错，如果发现错误则重传；

PS. 模2运算除法
$$\begin{array}{r} 10 \\ 10\overline{)\,101} \\ \underline{10} \\ 01 \\ \underline{00} \\ 1 \end{array}$$

第3章、运算方法和运算部件

1．行波进位加法器：将多个 1 位全加器串联，串行进位，速度慢；
进位选择加法器：类似分支预测，硬件较多
先行（超前）进位加法器：通过“进位生成”和“进位传递”函数使各进位独立，并行产生，速度快，可用单级，两级或更多级先行进位方式连接；

2．算术逻辑单元 ALU，在先行进位加法器基础上增加其它功能，实现基本算术和逻辑运算，有 2 个操作数输入，低位进位输入，操作控制输入（操作码位数决定了操作类型数量），1 个结果输出，1 位高位进位输出和相应标志输出等；
如 74181 是一种典型的 4 位 ALU IC，操作码有 4 位，加 1 位功能选择，可实现 16 种算术和 16 种逻辑运算；74182 是一种成组先行进位生成 IC，可配合 74181 扩展为多位两级先行进位（组内，组间均先行）加法器；



3．定点整数运算，由专门的定点运算器实现，核心部件是带快速加法器的 ALU；
移位运算：算术移位，逻辑移位，循环移位；
扩展运算：零扩展，适用于无符号数，高位补 0；符号扩展，适用于有符号整数（补码），高位补符号；
补码加减运算：符号位和数值位一起参加运算，减法用加法实现；同号相加时，**若结果的符号不同于加数的符号，则发生溢出**；加减法器，在基本加法器基础上增加进位/加减控制，求补电路，溢出判断电路等；
补码乘法运算：用加法和右移实现，符号位和数值位一起运算，采用 Booth 算法或 MBA 算法；快速乘法器，可用基于 CSA 的阵列乘法器，MBA+WT 乘法器实现；
补码除法运算：用加减法和左移实现，符号位和数值位一起运算，有恢复余数法和不恢复余数法（常用，加减交替，最后一次还是要恢复余数的）两种；阵列除法器，用可控加减单元 CAS 组合成一个除法阵列；

4．浮点数运算，由专门的浮点运算器实现；
加减运算：
1）对阶：向阶码**大**的对齐，阶码小的**尾数右移**，右移时保留**附加位**（指为保证精度，计算过程中多保留的位）；
2）尾数相加减：隐藏位和附加位都参加运算；用**定点小数原码加减法运算**实现，符号位判断操作，数值位参加运算，同号相加，异号相减，大数减小数，减法即加负数的补码，结果取大数的符号；
3）规格化处理：根据尾数结果进行左规或右规操作；
4）舍入：可采用就近舍入，正向舍入，负向舍入，截断 4 种方式；
5）溢出判断：当阶码上溢时，结果溢出；
乘除运算：
尾数用**定点小数原码乘除运算**实现，符号位，数值位分开运算，数值位用无符号数乘除法实现；
阶码用定点整数加减运算实现；

第4章、存储器分层体系结构★

1．存储器按**存取方式**分为：随机存取，顺序存取，直接存取，**相联存储**（按内容，如快表 TLB）
主要指标：容量，速度，价格；层次化结构；
SRAM — 6 管双稳触发器，MOS 管多，集成度低，功耗大，无需刷新，读写速度快，常作为 cache
DRAM — MOS 管的栅极电容，集成度高，功耗小，需定时刷新，读后再生，速度较慢，常作为主存
大容量存储芯片为减少地址引脚，多采用复用技术，行地址和列地址通过相同的管脚先后输入；
DRAM 的刷新是读出后原样写入，按行进行；
集中刷新；分散刷新（分散到每个存储周期）；异步刷新（常用，刷新间隔=刷新周期/行数）；
传统的内存与 CPU 之间采用异步方式交换数据；SDRAM，DDR 等是采用同步方式，支持突发传输；

一般采用海明校验；内存包括 RAM 和 ROM；**主存芯片内采用行缓存，当前行的数据直接从缓存中取；**

2．存储芯片的位扩展和字扩展；与 CPU 的连接：控制线（CS，W/R̄，行/列选通）；数据线；地址线；
多模块存储器，**连续编址**时 CPU 地址线的高位作为片选信号，低位表示模块内/体内地址，地址在模块内连续，CPU 串行访问存储模块，但 DMA 可同时访问其它的模块；**交叉编址**则以低位作为片选信号，高位表示模块内地址，多采用轮流启动方式，存取速度提高 n 倍，提高了带宽；
双口 RAM，发生冲突时可按优先级选择某一端口，也可增加一个 busy 引脚，避免冲突；

3．程序访问的局部性原理：时间（某单元在很短时间内被重复访问）/空间（相邻单元）局部性；Cache 的有效位；
Cache 平均访问时间$T_a = p \times T_c + (1-p) \times T_m$；$p$ 为命中率，适用于 cache 和内存并行访问的情况；
cache 行↔主存块，一行可能有 1 到多个字节，为 2 的整数幂；分块太小不能充分利用空间局部性，也使块数增加，冲突概率增大；若分块太大，在 Cache 容量一定的情况下，行数减小，导致缺失率增加，也会降低性能；

4．Cache 与主存之间的映射：
直接映射 — 每个主存块对应固定的 cache 行，行号=主存**块号** % cache 行数，若 cache 有 2^n 行，则主存**块号**（**不是主存地址！**）的低 n 位对应 cache 行号，主存地址形式：| 标记 | cache 行号 | 块内地址 |；首先按主存块号低 n 位找到对应 cache 行，cache 每行有对应的多位标记，若与主存标记相同，且 cache 有效位为 1，则命中；实现简单，命中时间短，无需替换算法，但不能充分利用 cache 空间，命中率低，适合大容量 cache；
全相联映射 — 每个主存块对应 cache 的任一行，主存地址形式：| 标记 | 块内地址 |；按内容访问，需要比较所有 cache 行的标记，时间和元件开销大，实现困难，冲突概率低，**不**适合大容量 cache；
组相联映射 —将 cache 调整为二维逻辑结构，组间模映射，组内全映射，组号=主存**块号** % cache 组数，主存地址形式：| 标记 | cache 组号 | 块内地址 |；cache 每组 s 行即称为 s 路组相联，适当选取 s，性能上就能接近全相联方式；首先按主存块号低 n 位找到对应 cache 组，对应 cache 组中每行标记与主存标记比较，若某行相同，且有效位为 1，则命中；

5．Cache 中主存块替换算法：先进先出（FIFO），缺失率较大；最近最少使用（LRU，是堆栈算法，每行需要计数器，命中率随组增大而提高）；最不经常用（LFU）；随机替换；

6．Cache 一致性问题，关键是处理好写操作：
全写法（Write Through）— 若命中，同时写 Cache 和主存；若不命中，写内存后，可以装入 cache，也可以不装入；
写回法（Write Back） — 若命中，只写 Cache，每行增加 dirty 位，被换出时才写内存；若不命中，先装入 cache，只写 Cache；需要其它机制保证 Cache 和内存一致性；
写一次法（Write Once）— 与**写回**法基本相同，仅是**第一次**写命中时要同时写 Cache 和主存；

7．Cache 性能，即命中率，与容量，行大小有关；现在一般采用多级 Cache，且 L1 分指令和数据两个 Cache；一般 L1 更关注速度，L2 则要求尽量高的命中率；采用快速查找算法，采用并行判定也能提高命中率；

8．虚拟存储见操作系统对应部分；

9．快表（TLB）通常采用全相联或组相联，随机替换算法，写回法；

10．在一个具有 Cache 和虚存的系统中，CPU 访存最长的过程是：CPU 给出虚拟地址 VA→TLB 缺失→访问主存页表→缺页→主存没有空闲页，需换页→从硬盘读出一页到主存→更新页表和 TLB→将 VA 转换为物理地址 PA→Cache 缺失→Cache 中没有空闲行，需替换→主存块送入 Cache→读取数据；

11．增加 Cache，是为了匹配 CPU 和内存的速度；虚存主要是为了解决内存容量有限的问题，同时也解决了进程内存空间的隔离保护和共享问题；

第5章、指令系统

1．指令**系统**的设计原则：完备性；兼容性；均匀性（能处理多种数据类型）；可扩充性；
指令=操作码[+地址码]（不一定是地址）；指令一般为整数个字节，可能全部指令长度固定，也可以操作码长度固定，或指令和操作码都固定，也可以都不固定；
定长— 方便地址计算、取指和译码；变长（扩展）—紧凑，节约存储空间；
零地址指令或者是无操作数，不需要地址，或者操作数隐含在堆栈中，地址由栈指针给出；
寄存器数量多，可减少访存，提高性能，但会增加指令长度，增加芯片成本；

2．操作数类型：地址/指针，用无符号整数表示；有符号整数，浮点数，十进制数；字符串，逻辑值；

3．寻址方式
1）立即（数）寻址，ADD #FFH；2）直接寻址，MOV AL, FFH；3）间接寻址（指针），MOV AL, [FFH]；
4）寄存器寻址（寄存器本身）ADD AL；5）寄存器间址 MOV EAX, [ECX]，可扩大寻址范围；
6）变址寻址，MOV AL, [SI*Size+1000H]，常用于循环中对数组访问，1000H 是数组首地址，SI 是循环变量，Size 是

数组元素的大小；有的计算机还允许变址和间址结合，EA=((I)+A)为先变址再间址，称为前变址，是一个指针数组；EA=(I)+(A)为先间址后变址，称为后变址，A 为指针的指针；

7）相对寻址，用于指令寻址，地址码给出的是偏移量，是有符号整数，基址隐含由 PC 给出，实现指令跳转或浮动子程序，JMP FFH；

8）基址寻址，EA=(Base)+A，在寻址过程中，基址是固定的，A 可变，用于进程的重定位/分段；变址，基址和相对三种很类似，统称偏移寻址；

9）其它：隐含累加器寻址；堆栈操作，隐含在栈顶；位寻址；块寻址，串寻址，首地址+长度/末地址/尾标志；

4．操作类型

数据传送　　：在寄存器，主存，栈顶之间传送数据；
运算　　：各种算术和逻辑运算；
移位　　：算术/逻辑左移，低位补零；算术右移，高位补符号；逻辑右移，高位补零；半字交换等；
串处理　　：串查找，传送，比较等；
I/O 操作　　：CPU 与外设接口进行数据、状态、命令交换；
程序流程控制：条件转移，无条件转移，程序调用，返回等；
系统控制　　：停机，中断，模式切换，开关中断，空操作等，一般为特权指令；

5．状态位/条件码/标志位(Flags)

1）符号位 NF/SF = 1 表示结果为负数；　　2）溢出位 VF/OF = 1 表示结果溢出；
3）进位/借位标志 CF = 1 表示产生了进位/借位；　　4）零位 ZF = 1 表示结果为 0；
条件转移指令会用到这些标志位，如无/有符号数是否为 0 或另一个数的大小关系（做减法）；
对无符号数：a==b ↔ZF；a>b↔ $\overline{CF+ZF}$ ↔ $\overline{CF}\cdot\overline{ZF}$ ，a≤b↔ $CF+ZF$；a<b↔$CF\cdot\overline{ZF}$ ，a≥b↔ $\overline{CF}+ZF$ ；
有符号数还要考虑溢出位 OF；

6．按地址码指定风格，指令系统有累加器型（早期），堆栈型（JVM，CLI），通用寄存器型（x86），Load/Store 型（也算是一种通用寄存器型，但限制访存方式，RISC）；

7．按指令复杂度，指令系统有

CISC — a.指令系统复杂，指令多，寻址方式多，指令格式多；b.指令周期长；c.指令周期差距大；d.采用微程序控制；e.编译优化**较难**；

RISC — a.指令数量少，只含常用的简单指令；b.指令格式规整；c. Load/Store 型风格；d.流水线方式执行指令；e.大量通用寄存器；f.采用组合逻辑电路控制；g.有利于编译优化；

RISC 随着系统功能扩展，日趋复杂；CISC 采用微指令，吸收 RISC 的优点，也实现了流水线，又保持了指令集兼容；

第6章、中央处理器★

1．CPU 的执行指令的过程：1）指令地址计算；2）取指令，对定长指令，通常只需一次访存，若指令长度可变，则可能需多次访存；3）指令操作码译码，生成控制信号；4）源操作数地址计算，取操作数；5）数据运算；6）目的操作数地址计算，存数；7）检查是否发生中断，如有则转入中断处理程序；

2．CPU 的基本功能：1）控制指令的执行顺序；2）控制指令执行操作（what）；3）控制操作时序（when）；4）对数据进行运算；5）对存储器或 IO 访问进行控制；6）异常和中断处理；

3．CPU 基本结构：由**数据通路**和**控制单元**组成；

数据通路：包含组合逻辑单元和存储信息的状态单元；

组合逻辑单元用于对数据进行处理，如加法器，ALU，扩展器（0 扩展和符号扩展），多路选择器，以及状态单元的读操作逻辑等；

状态单元包括触发器，寄存器等，用于保存指令执行的中间状态或最终结果；某一时刻的输出状态可能与输入端不同，再经过一个时钟边沿后，输入状态才能传递到输出端；

早期有累加器+内存结构的数据通路（如 IAS），还有采用总线结构的数据通路（单总线，多总线，总线与部件间一般要通过寄存器和三态门隔离），现代多采用专用数据通路结构；

控制单元：对取出的指令译码，与指令执行得到的条件码，或当前机器状态，时序信号等组合，生成对数据通路的控制信号；包括 PC，IR，指令译码器，时序信号产生部件，操作控制信号形成部件，总线控制逻辑，中断机构等；

4．用户可见寄存器：用户程序中的指令可直接访问或间接修改的寄存器；有，

通用寄存器：用来存放数据或地址；
地址寄存器：专门用来存放首地址或指针，如段寄存器，变址寄存器，基址寄存器，堆栈指针等；
程序计数器 PC/EIP：存放下一条指令的地址；

部分可见寄存器：只能读取部分信息的寄存器，如程序状态字 PSWR 或标志寄存器，其内容由 CPU 根据指令执行结果自动设置，用户程序只能隐含读出部分内容，不能直接修改这些寄存器的内容；

不可见寄存器：用于记录控制和状态的寄存器，只能由 CPU 硬件或**操作系统内核**访问，用户程序不可访问；

指令寄存器 IR：存放正在执行的指令；

存储器地址寄存器 MAR：存放将要访问的存储单元地址；

存储器数据（缓冲）寄存器 MDR：存放从存储器读出或要写入存储器的数据；

其它寄存器：如中断请求寄存器，进程控制块指针，页表基址指针等；

5．早期计算机采用机器周期，节拍，脉冲三级时序系统；现代计算机没有机器周期的概念，而一个节拍即一个脉冲；数据通路采用时钟信号进行定时，在时钟边沿，数据通路中的状态单元开始写入信息；

每条指令功能不同，所以执行时数据在数据通路中经过的部件和路径可能不同，但取指阶段是一样的；

6．控制器的功能和控制原理；

7．控制单元的实现方式：硬连线路控制器（组合逻辑控制器）和微程序控制器；

硬连线路控制器：将指令执行过程中每个时钟周期所包含的控制信号取值组合看成一个状态，每来一个时钟，控制信号会有一组新的取值，也就是一个新的状态，这样所有指令的执行过程可以用一个有限状态转换图来描述；用一个组合逻辑电路（一般为 PLA）来生成控制信号，用一个状态寄存器实现状态之间的转换；速度快，适合简单或规整的指令系统，如 RISC；

微程序控制器：将指令执行过程中每个时钟周期所包含的控制信号取值组合看成是一个 0/1 序列，每个控制信号对应一个**微命令**，控制信号取不同的值，就发出不同的微命令；**若干微命令组合成一个微指令**，每条指令所包含的动作由若干条微指令来完成；每条指令执行时，先找到对应的第一条微指令，然后安装特定的顺序取出后续微指令执行；每来一个时钟脉冲，执行一条微指令；实现时，每条指令对应的微指令序列实现存放在一个只读存储器（称为控制存储器，简称控存 CM）中，用一个 PLA 或 ROM 来生成每条指令对应的微程序的第一条微指令地址，用相应的微程序定序器来控制微指令执行过程；结构标准化程度高，便于扩充，速度较硬连线结构慢；

微程序定序器（即确定下条微指令地址的部件）的实现有计数器法（μPC）和断定法（又称下址字段法，显式给出下条微指令地址）；

8．微指令的编码方式：

直接控制法：在微指令的控制字段中，每一位代表一个微命令，在设计微指令时，是否发出某个微命令，只要将控制字段中相应位置成“1”或“0”，这样就可打开或关闭某个控制门；全 0 对应 nop 指令；微操作码长度与微命令个数相同，无需译码，控制电路简单，速度快，便于并行执行；但浪费码位；

字段直接编译法：在计算机中的各个控制门，在任一微周期内，不可能同时被打开，而且大部分是关闭的（相应的控制位为“0”）；所谓微周期，指的是一条微指令所需的执行时间；如果有若干个（一组）微命令，在每次选择使用它们的微周期内，只有一个微命令起作用，那么这若干个微命令是**互斥**的；选出互斥的微命令，并将这些微命令编成一组，成为微指令字的一个字段，用二进制编码来表示，就是字段直接编译法；**相容**微命令在不同的字段里；

字段间接编译法：在字段直接编译法的基础上，进一步缩短微指令字长的一种编译法；在字段直接编译法中，还规定一个字段的某些微命令，要兼由另一字段中的某些微命令来解释；极少采用；

9．异常（内中断）指 CPU 内部在执行某条指令时发生的程序异常或硬件异常，有故障，陷阱和终止三种类型，也被称为程序性中断；

故障：由某条正在执行的指令产生的异常，如溢出，除数为 0，非法操作，缺页等，有些故障处理后程序可以继续执行，有的不能处理，只能停止执行发生异常的进程；

陷阱：预先安排的一种“异常”事件，如断点，单步跟踪，系统调用等；

终止：严重的硬件故障，一旦发生只能终止整个系统，或者重启；

外部中断，外设或其它 CPU 通过中断请求线向 CPU 发出的处理请求，与当前指令无关，是一种 IO 控制方式；

10．异常处理过程：1）保护断点和程序状态（PC 和 PSW）；2）识别异常事件并转入处理；3）关中断；

第7章、指令流水线★

1．指令流水线的基本概念：

流水线技术是一种能显著提高指令吞吐率的技术；方法是指令取指后，不等该指令执行完毕，即取下一条指令；如果把一条指令的解释过程进一步细分，例如，把分析、执行两个过程，分成取指 IF、译码 ID、取数 OF、执行 EX、和写回寄存器 WB 5 个子过程，并用 5 个子部件分别处理这些子过程；在上一指令的第 1 子过程处理完毕进入第 2 子过程处理后，第 1 子部件中就开始对第二条指令的第 1 子过程进行处理；随着时间推移，这种重叠操作最后可达到 5 个子部件同时对 5 条不同指令的子过程进行处理；

将每条指令的执行规整化为若干个流水阶段，**每个流水阶段的执行时间以最慢的流水段所需时间为准，定为一个时钟周期**，这样可能有的指令执行时间变长了，但在理想情况下，每个时钟有 1 条指令进人流水线，并有 1 条指令执行结束，达到 CPI=1，最终的效果是执行效率显著提高了；

每个流水段中的部件都是组合逻辑部件，流水线段之间需要加流水段寄存器，组合逻辑中产生的结果在时钟到来时被存储到流水段寄存器中；流水段寄存器用以记录所有流到后面阶段要用的各种信息，例如控制信号、指令、新的 PC 值、参加运算的操作数、指令运算结果、指令异常信息、寄存器读口地址、寄存器写口地址、存储地址等；

指令译码得到的控制信号通过流水段寄存器与本指令的数据信息一起，同步传送到后面各个流水段；

2．指令流水线的局限性：

不同指令的功能不同，并不是每条指令都能划分成相同多个阶段，按最复杂指令所需规划流水段后，有些指令的某些流水段执行的可能是空操作 nop；

不同流水阶段的功能不同，并不是每个流水段所用的时间都一样长，按最长时间流水段确定时钟周期后，某些流水段中可能会有时间浪费；

随着流水线深度的增加，流水段寄存器的额外开销比例也增大；指令在资源冲突、数据相关或控制相关时会发生流水线阻塞，因而影响指令执行效率；

3．指令流水线的执行效率

吞吐率：比非流水线方式下提高若干倍，理想情况下，其倍数为划分的流水段个数；

加速比：指 m 段流水线的速度与等功能的非流水线的速度之比，理想情况为 m 倍；

指令执行时间：由于流水段划分要求的一致性，以及流水段寄存器的额外开销，使得流水线方式下一条指令的执行时间更长了；

4．**结构冒险（资源冲突）**：多条指令同时要求使用同一个功能部件；解决策略：

规定每个功能部件在一条指令中只能被使用一次；

规定每个功能部件只能在某个特定的阶段被使用；

指令存 cache 和数据 cache 分开；

5．**数据冒险（数据相关）**：前面指令的目的操作数是后面指令的源操作数；解决策略：

用软件（如编译器）在数据相关指令前插入 nop 指令；

在硬件检测到数据相关时，使后面的数据相关指令进入停顿状态，即在特定的流水段中插入“气泡”，以“阻塞”指令继续执行，直到取得所需数据为止；

利用“**转发（旁路）**”技术，把前面指令执行过程中得到的数据直接传送到后面指令需要使用数据的地方，能解决**写后读（RAW）**，或**写后写（WAW）**情况；

对于**取数后直接使用**的情况（如 Load 指令取出的数据是随后下一条运算指令的操作数，load-use 情况，又可归为**读后写，WAR**），则采用“**阻塞加转发**”的方式解决数据冒险；

6．**控制冒险（控制相关）**：有两种情况，

1）转移指令：分支、调用、返回、跳转等指令（统称为转移指令）需要计算目标转移地址，分支指令还需根据条件确定新的 PC 值；由于获取转移目标地址的时间较长，使得在目标地址产生前，后续指令已被取到流水线中；如果已经取出并正在执行的指令不是应该执行的指令，则发生了控制冒险；分支指令引起的控制冒险称为分支冒险；

2）异常和中断：处理器检测到发生异常和中断时，可能已有多条不该执行的指令在流水线中执行，导致控制冒险；

解决策略：

由**编译器**在转移指令后面插入 nop 指令（插入指令条数等于延迟损失时间片）；

在硬件检测到有关转移指令时，使后面若干条指令（指令条数等于延迟损失时间片）进入停顿状态，也即在特定的流水段中插入“气泡”，以“阻塞”指令继续执行，直到能确定正确的 PC 值为止；

对于**分支冒险**，可结合采用“**分支预测**”技术；有简单（静态）预测和动态预测两种方式；静态预测方式下，预测的结果是固定的，总是预测转移（taken）或不转移（not taken）；动态预测方式下，根据分支指令执行的历史情况调整预测位；动态预测比静态预测成功率高得多；

采用延迟分支技术；将前面几条与转移指令无关的指令放到分支指令后面的延迟槽中执行，无关指令不够时用 nop 指令填满延迟槽；这样，硬件不需对流水线进行阻塞，对指令顺序进行调整的工作在编译阶段完成；

7．在一个时钟周期内，流水线的每个阶段都可能发生异常或中断，一般的做法是只将异常原因和断点记录到特定寄存器，指令继续在流水线中执行，**直到最后阶段**，检测该指令是否发生了异常或出现外部中断，若有则刷新流水线并转入中断处理过程；

8．高级流水线技术：在时间和空间上进一步提高指令级并行性，通常有三种措施：

超流水线：将指令执行过程划分得更细，采用更多级数的流水线，着重在于**时间**上并行；

多发射流水线（超标量处理器）：同时发射多条指令，并由多个功能部件并行执行，着重在于**空间**上并行，CPI<1；“指令打包”和“冒险处理”是实现多发射的两个基本任务；

静态多发射：“指令打包”和“冒险处理”任务主要由编译器静态、完成，打包后的指令相当于一条由多条指令组成的长指令，被同时发射执行，因此，这类处理器有时被称为 VLIW 处理器（超长指令字）；Intel 的 IA-64（Itanium）架构采用这种方法，Intel 称其为 EPIC 技术（显式并行指令运算）；过于依赖编译器，硬件的变更会导致不兼容，甚至需重新编译应用软件；

动态多发射：由处理器硬件在指令执行时动态确定哪些指令被同时发射，如果没有可以被同时发射的指令或遇到指令相关时，则某些流水段空闲；目前超标量处理器多指采用动态多发射流水线的处理器，并且通常会结合采用动态流水线调度技术；

动态流水线调度：处理器通过指令相关性检测和动态分支预测等手段，投机性地不按指令顺序执行；即当发生流水线阻塞时，根据指令的依赖关系，动态地到后面找一些没有依赖关系的指令提前执行；在动态流水线调度下，指令被“乱序”执行；

第8章、系统总线

1．总线是一组设备共享的传输线，一次只能为一个部件提供服务，其发出的信号可以其它所有部件接收；
2．按连接部件可分为内部总线（片内），系统总线（功能部件），通信总线（主机与 IO 或其它主机，如 USB，以太网）；
3．系统总线按传输信息，可分为数据总线（双向传输），地址总线（单向），控制总线（用来传输时钟和命令信息）；也有将地址线和数据线复用，经数据线传送地址的设计；
4．对同步总线，带宽 B 与**数据**总线位宽 W，总线时钟频率 F 和传输每个数据所需的时钟周期 N 有关，B=W・F/N；
5．总线上一对设备间一次交换信息的过程称为总线事务，典型如存储器读写，IO 读写，中断响应等；一次总线事务包括地址阶段和数据阶段，详细又分申请分配阶段，寻址阶段，数据传送阶段和结束阶段；突发（burst）传送由一个地址阶段和多个数据阶段构成，可传送多个连续单元的数据，也称为组传送；
6．一个总线事务总是在一对主，从设备之间传送数据，当有多个主控设备时，需要总线裁决来决定哪个主控设备使用总线；有集中式（有总线控制器/裁决器）和分布式两种裁决方法；
7．集中式 — 主控设备使用请求信号发出请求，获准后传送，完成后需通过某种方式通知总线控制器，释放总线；

链式查询/菊花链：只有总线请求，允许，忙三根控制线（前 2 根用于裁决），各主控设备串接在允许线上，被选中设备置总线忙信号；靠近控制器的设备优先级高，结构简单，但不能保证公正，可靠性低，裁决速度慢；**击鼓传花**

计数器定时查询：n 个设备，有 ceil(log2(n))根设备地址线，一根请求线，和总线忙状态线，裁决器内的计数器即选中设备的地址，被选中设备置总线忙信号；若每次裁决都从 0 开始计数，则设备地址即优先级（越小优先级越高），若从上次选中设备开始，则优先级平等；也可由程序设置计数初值，从而调整优先级；可靠性较高，但控制较复杂；**点名报到**

独立请求方式：每个设备都有一根请求线和允许线，共 2n 根控制线，没有总线忙状态线；裁决器内有排队器，优先级控制方法非常灵活，裁决响应快，但控制逻辑和结构复杂；**领导说了算**

8．分布式 — 没有中央的裁决器，每个设备有自己的裁决器和请求号，竞争使用总线，自行裁决；

自举式：每个设备一根请求线，兼做总线忙状态线，检测较高优先级设备的请求线均为空闲时可使用总线；优先级固定，请求线较多，故多复用数据线；

冲突检测式：如以太网的载波监听多点接入/冲突检测（CSMA/CD）方式；控制线少，冲突多；

并行竞争式：n 个设备，有 ceil(log2(n))根仲裁线，各设备**出牌（仲裁号）比大小**，设备和仲裁线的逻辑相反；优先级灵活，裁决线少；

9．总线的定时方式有 4 种：同步，异步，半同步和分离事务；

同步：总线上的所有设备从公共的时钟线上获得定时信号，传输协议规定了在每个周期应完成的特定操作；传输速度快，接口逻辑少；但必须按最慢的设备来确定时钟，适合于存取时间比较接近的设备；由于时钟偏移，导致总线不能过长；由于没有上一步操作的确认信号，故可靠性较低；

异步：采用应答（握手）方式，需增加请求和应答线；有不互锁，半互锁和全互锁（三次握手）三种应答方式；

半同步：在异步方式中增加时钟线，要求握手信号总是在时钟边沿发生，提高了抗干扰能力；

分离事务：设备不使用总线时立即释放，多个设备可交替使用总线，但完成一个事务可能要多次获得总线，从设备也必须跟踪对方的状态，与主设备联系；

10．总线标准：系统总线 ISA，EISA，PCI，PCI-E 等；设备总线/接口 IDE，SATA，SCSI，USB 等；图形总线/接口 AGP 等；
11．总线结构：**单总线，双总线**（主存总线+系统总线，或主存总线+IO 处理器& IO 总线），**多总线**（IO 总线+主存总线+高速 IO 与内存之间的 DMA 总线，处理器总线+桥接器+主存总线+高速 IO 总线+ IO 处理器&低速 IO 总线）；

第9章、输入输出组织

1．**输入**设备（键盘，鼠标等），**输出**设备（显示器，打印机等）和**外部存储器**（硬盘，光盘，U 盘，磁带等）统称外设；外设的特点有异步性，实时性和多样性；

2．硬盘及 RAID 见操作系统对应部分；

3．磁表面存储器的数据记录方式

归零制（RZ）：正脉冲为 1，负脉冲为 0，各 bit 之间电流总归零，实为±1，0 三态，记录密度较低，有自同步能力；

不归零-1 制（NRZ-1）：电流翻转为 1，各 bit 间无间隙，记录密度较高，无自同步能力，直流分量大，功耗较大；

调相制（PM）：利用电流跳变，由负到正↑为 1，由正到负↓为 0，每个 bit 都有一次电流换向，可生成同步脉冲，实现自同步；**即曼彻斯特编码**

调频制（FM）：各 bit 间一定有电流跳变，bit 内有跳变为 1，无跳变为 0，即 1 的频率为 0 的 2 倍，有自同步能力；

改进调频制（MFM）：连续两个或以上的 0，在**位边沿**才有电流跳变，bit 内有跳变为 1，无跳变为 0（实际是以两位 00,01,10,11 来设计记录模式）有自同步能力，记录密度提高，**称为倍密度，应用广泛**；

4．IO 接口的功能：数据缓冲，错误或状态检测，控制和定时，数据格式转换，与主机和设备通信；

IO 接口按数据传送方式有串口/并口；按功能的灵活性有可编程/不可编程；按通用性有通用/专用；按设备连接方式有总线多点/点对点；按数据传送的控制方式有程序控制（轮询）/中断式/DMA；

IO 端口指 IO 接口中的用户可访问寄存器，有数据端口，状态端口和命令端口，后面两者传送方向相反，被访问的时间一般也是错开的，故经常复用；IO 端口编址方式有独立编址和统一编址两种；

5．通常并行传输多用在同步总线中，但这不是必须的；串行传输多用在异步或半异步总线中，也可用于同步总线；

6．并行方式的一次可传送多位数据，但各数据线间存在干扰，而且各数据线间长度或传输特性偏差会使传送的**位不同步**，当提高总线时钟频率时，这种位不同步的可能性更大，这限制了并行总线不能过长，时钟频率也受到限制，最终限制了带宽；而串行总线没有位同步问题，若采用差分方式，抗干扰能力强，传输距离长；还可以充分提高总线时钟频率，弥补位宽的不足，在某些场合优于并行总线；

系统总线有多根数据线，是并行的；USB 是通信总线，是串行（差分方式）的，以太网使用的双绞线也是串行的；

7．IO 控制方式分为程序直接控制，程序中断，直接存储器存取（DMA），通道和 IO 处理机 4 种方式；

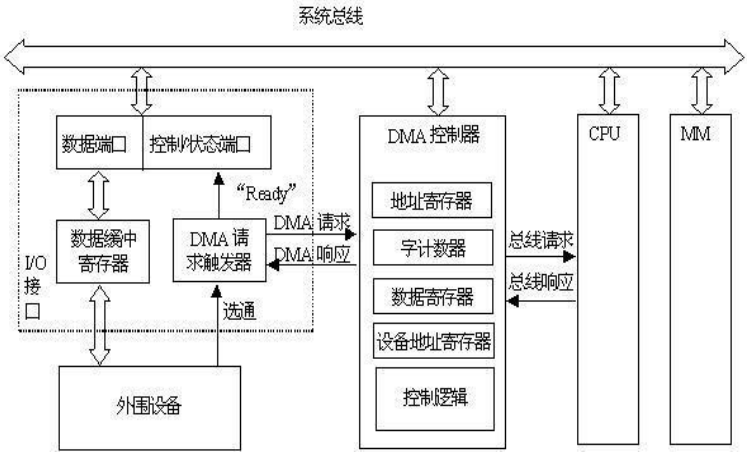
8．**程序直接控制：无条件传送方式/同步**（利用程序定时传送数据，无需检测接口或设备的状态，适合过程控制和巡回检测）；**条件传送/程序查询方式/异步**（通过查询外设接口的状态来控制数据的传送，有定时查询和独占查询）；

9．程序中断：外设通过发送中断请求通知 CPU，在中断服务程序中完成数据/错误处理；

中断系统有响应优先级和处理优先级两种优先级；响应优先级是由查询程序或硬件判优排队决定的，多个中断同时请求时选择哪个响应；处理优先级是由中断屏蔽字来动态设定的，反映了多重中断直接的关系；中断是异步的；

10．**直接存储器存取（DMA）**：适合磁盘等高速外设以成批方式和主存直接（双向）交换数据；

首先对 DMA 控制器初始化，然后又 DMA 控制器控制总线在主存和高速设备之间直接交换数据，无需 CPU 干预；最后由 DMA 控制器发出“DMA 传送结束”信号给**外设接口，并发中断请求**给 CPU，CPU 进行 DMA 结束处理；协调 DMA 与 CPU 访存冲突有三种方法：CPU 不访存；周期挪用（CPU 让给 DMA，**DMA 总线优先级比 CPU 高，以防止高速设备丢失数据；每次访存都要申请总线**）；交替分时访问；



中断响应在一个指令周期结束后；而 DMA 响应是在一个总线周期后；

DMA 导致的虚存跨页和 Cache 一致性问题；

11．**通道和 IO 处理机**：多使用 CPU—通道—设备控制器—IO 设备四级连接方式；通道有专用的通道指令，可独立地执行这些指令完成 IO；由 CPU 的 IO 指令启动，停止或改变工作状态，是从属与 CPU 的一个专用处理器；按数据传送方式，通道有字节多通道，选择通道，成组通道（有人称为数组通道）；

字节多通道：按**字节**交替传送数据，适合连接多个低速 IO 设备，数据传输率是所有设备传输率之和；

选择通道：适用于高速设备，一段时间内只为单个设备服务，采用成组方式传送，传输率为所接设备最大的传输率；

成组通道：所接多台设备以定长数据块分时传送数据，传输率为所接设备最大的传输率；

IO 处理机与通道相比，有自己的指令系统和单独的存储器，有一定的数据处理功能；

操作系统

《操作系统教程 第4版》孙钟秀 高等教育出版社 ISBN 978-7-04-023221-9

第一章、操作系统概述

1．讨论操作系统的4种观点：1）用户接口与公共服务程序：2）程序的控制者和协调者；3）扩展机与虚拟机；4）资源管理者与控制者；
操作系统资源管理功能主要有：处理器管理、存储器管理、设备管理、文件管理、网络与通信管理；为了实现资源管理，主要使用资源复用，虚化和抽象技术；通过进程抽象，虚存抽象和文件抽象来创建一个虚拟机；
2．操作系统是一个大型复杂的并发系统，并发性、共享性、随机性（异步性）是它的重要特征；
3．通道技术的引进和中断技术的发展，导致了操作系统由单道作业处理进入了多道程序设计系统阶段；实现多道程序设计必须妥善处理：1）存储保护与程序浮动；2）处理机的管理与分配；3）资源的管理与调度；
这一阶段，操作系统沿着三条主线发展：多道批处理系统、分时交互系统和实时处理系统；
多道批处理系统着眼于让处理器和外围设备同时保持忙碌，提高作业的吞吐率和整个系统的效率；其关键机制是：在响应一个作业的处理结束信号时，处理器将在主存中驻留的不同作业间切换；
分时交互系统的主要设计目标是为用户提供方便的程序开发、调试环境和快速响应交互式用户的命令请求，但又要支持多用户同时工作，以降低系统的成本；由于用户键盘操作速度较慢，快速响应用户请求和同时支持多用户这两个目标是可以共存的；其关键机制是：采用时间片轮转法，让处理器在多个交互式用户间多路复用；
与分时系统相比，实时事务系统常局限于一个或几个应用，例如，数据库的查询和修改应用或生产过程控制实时应用，有响应时间的限制；其关键机制是：事件驱动机制，当系统接受来自外部的事件后，快速分析这些事件，驱动实时任务在规定的响应时间完成相应处理和控制；
4．操作系统的基本类型有三种：批处理系统、分时系统和实时系统；凡具备全部或兼有两者功能的系统称通用操作系统；随着硬件技术的发展和应用深入的需要，新发展和形成的操作系统有：微机操作系统、网络操作系统、分布式操作系统和嵌入式操作系统；
5．操作系统的结构：单内核（整体式结构、层次式结构）、微内核结构；内核是由中断驱动的，即只有在中断或异常发生后才会引出内核工作，处理完成后内核就会退出；内核与裸机为每个进程形成了一台没有中断的虚拟机，进程独占虚拟的CPU，内存和外设，不需要处理中断；

第二章、处理器管理

1．操作系统是由中断激活的，中断装置识别并响应中断事件，通过交换PSW，让中断处理程序占有处理器；在处理完该中断事件后，通常会改变一些进程的状态，引起相应进程队列的调整，然后转向低级调度执行调度工作；中断能改变处理器执行的顺序，是操作系统实现并发性的硬件基础之一；中断也是用户态向核心态转换的唯一途径；
2．为了屏蔽中断的影响，操作系统较早地引入了一个由并发执行的顺序程序组成的概念模型—进程，通过中断方式进程可以在实处理器上交替执行，每个进程可以认为都运行在自己的一个虚拟处理器上；
3．进程是程序的执行，是一个动态的概念；进程映像是某时刻进程的内容与状态的集合，包控：进程控制块（PCB），程序，数据（私有栈等），核心栈；进程上下文是处理器的一个执行环境，包括：用户级上下文（程序，数据，私有栈等）、系统级上下文（PCB，页表，核心栈等）、寄存器上下文（PSW，PC，SP，控制寄存器，通用寄存器等）；
4．进程的三态模型：1）运行态；2）就绪态；3）等待态（又称阻塞或睡眠）；可能的状态转换及原因：
   1) 运行→等待：运行进程等待使用某种资源，或某事件发生，如设备传输或键盘输入；
   2) 等待→就绪：所需资源得到满足或某事件已完成；
   3) 运行→就绪：运行时间片到时，或出现更高优先级的进程，当前进程被迫让出CPU；
   4) 就绪→运行：调度进程选中当前进程执行；
进程在运行过程中执行系统调用，或发生中断时，操作系统获得控制权，当发现当前时间片耗尽，或执行一个**阻塞型**系统调用（如进程创建，退出等），就会导致进程切换和调度；
5．操作系统中引入线程后，进程涉及到资源所有权及其保护，线程是调度的独立单位，涉及到程序的执行；线程实现方法：用户级、系统级和混合式；
6．处理器调度有抢占式和非抢占式两种，按层次分，有高级调度，中级调度和低级调度；
高级调度又称作业调度，用来决定满足资源需求的后备作业选中进入主存去运行，控制多道程序的道数；
   1) 先来先服务，FCFS，用队列实现；
   2) 最短作业优先，SJF，非抢占式，平均周转时间最短，但长作业可能发生“饥饿”；

3) 优先级调度，既可用于抢占式，也可用于非抢占式；
静态优先级，定级方法：按进程类型（系统/用户，前台/后台），资源需求，用户类型等；
动态优先级，定级方法：已占用 CPU 时间长短，等待时间长短；
相同优先级的则可按 FCFS 或 SJF 调度，形成多级队列；

4) 高响应比优先：响应比=作业响应时间/估计运行时间=（等待时间+估计运行时间）/估计运行时间；有利于短作业，也考虑到了等待时间，不会导致长作业“饥饿”；

中级调度主要是根据主存大小决定进程数目，并决定与辅存的对换（Swap）；

低级调度又称进程/线程调度，决定哪一个进程或线程占有处理器运行，是各类操作系统必备的功能和最为核心的部分，执行十分频繁，调度策略的优劣直接影响整个系统的性能；

1) 先来先服务；
2) 最短进程优先；
3) 优先级调度；
4) 轮转调度，RR，使用时钟中断划分（均等或可变长）时间片；抢占式；时间片短则进程切换开销比率大，时间片长则接近 FCFS 的效果；
5) 多级反馈队列，MLFQ，又称反馈循环队列，是时间片轮转和优先级调度的结合；每个优先级对应一个就绪队列；高优先级分配的时间片**短**；通过对等待时间反馈来提高低优先级队列中进程的优先级，防止“饥饿”；
6) 彩票调度，彩票代表了资源的份额，容易实现按比例分配资源；

7．实时系统调度算法：单比率调度、限期调度、最少裕度调度；
8．多处理器调度算法：负载共享调度、群调度、专用处理机调度、动态调度；
9．衡量调度算法优劣的指标有：资源利用率、作业吞吐率、公平性（不会出现饥饿）、响应时间（分时/交互式，用户提交命令到得到响应）、周转时间（批处理，从提交作业到作业完成）等；

第三章、同步、通信与死锁

1．进程可并发执行的条件：1）$R1 \cap W2 = \emptyset$，且$R2 \cap W1 = \emptyset$，保证一个进程在读取存储器时，其中的数据不发生变化；2）$W1 \cap W2 = \emptyset$，保证写操作不会覆盖丢失；
2．并发进程之间存在两种基本关系：竞争和协作；
并发进程可能需要竞争使用资源，资源竞争可能导致死锁或“饥饿”；引入了**临界区**（并发进程中与共享变量有关的**程序段**）以解决进程互斥问题；临界区使用准则：**空闲让进，忙则等待，有限等待，让权等待**；

Perterson 算法：

```
// Perterson 算法是一种谦让算法，多个进程共享一组标志 inside[]，每个进程都可以读取其它进程的标志，
// 但只设置自己的标志，表示自己希望进入临界区（访问共享数据）
bool inside[2];
inside[0]=false;
inside[1]=false;

int turn;    //共享变量，指示哪个进程可以进入临界区，每个进程都把 turn 设置为另外一个进程，以示谦让
```

`void ProcessA(){`	`void ProcessB(){`
`    inside[0]=true;`	`    inside[1]=true;` `//B 希望进入临界区`
`    turn=1;`	`    turn=0;` `//谦让一下，先让 A 进入`
`    while(inside[1]&&turn==1);`	`    while(inside[0]&&turn==0);` `//若 A 想进且没轮到 B，则循环，等待`
`    //A 的临界区代码`	`    //B 的临界区代码` `//结束循环等待，可以执行临界区`
`    inside[0]=false;`	`    inside[1]=false;` `//退出临界区`
`}`	`}`

Perterson 算法是一种解决互斥的软件方法，其中的 while 语句相当于一个自旋锁 spinlock，此外还有硬件的方法，如关中断（单 CPU），锁总线（多 CPU），专用原子指令（XCHG）等；spinlock 虽然让 CPU 处于忙等状态，浪费了 CPU 时间，但与信号量方式相比，不会强制阻塞当前线程从而导致进程切换和调度，适合细粒度的场合；

同步是解决进程间协作关系的手段；进程同步的主要任务是使并发协作的进程之间能有效地共享资源和相互协作，协调各进程的进度，从而使进程的执行过程具有可再现性和执行结果的唯一性；

互斥是一种特殊的进程同步关系；同类进程一般是互斥关系，不同类进程一般是同步关系；

3. 进程间的低级通信机制一般只是传递信号，主要有：原子操作，锁机制，信号（Signal），信号量（Semaphore）和 PV 操作，管程等；使用低级通信机制解决许多经典同步问题的实例，如生产者与消费者问题、读者与写者问题、5 个哲学家吃通心面问题、睡觉的理发师问题，这些问题是计算机操作系统中并发进程相互制约和内在关系的一种抽象；从理论上来说，各种同步机制都是等价的，每种机制都可以用另一种来实现，但在实际应用中，信号量，消息传递和共享主存方式应用较多；
4. 信号量 Semaphore 是一种特殊的结构变量，代表了某个资源；

   Semaphore 包括一个有符号整数值 cnt，当 cnt 为正，表示可用资源数量，当 cnt 为负，表示等待（阻塞）该资源的进程数量；为此，还要有一个队列 queue 来记录这些等待进程；

   Semaphore 只允许两种操作，P（检测），V（增加），它们都是原子性的，保证对信号量本身的操作不会引起冲突；

   1) P 操作： cnt--，表示占用了一个资源，如果 cnt<0，则阻塞当前进程，并将其加入等待队列 queue 中，否则不特别处理，当前进程继续执行；
   2) V 操作： cnt++，表示释放了一个资源，如果 cnt<=0，则从等待队列 queue 中唤醒一个进程，将其转入就绪态，否则不特别处理，当前进程继续执行；
   3) 信号量解决互斥问题：一组同类进程，初始 cnt=1，进程执行 P 进入临界区，退出后执行 V 操作释放信号量；

```
semaphore mutex;                    // 信号量是由多个进程共享的
mutex =1;                           // 设置为 1 即可实现进程互斥
void ProcessA(){                    void ProcessB(){
    P(mutex);                           P(mutex);          //mutex-1，若 B 先执行 P，则阻塞 A，否则阻塞
    //A 的临界区代码                    //B 的临界区代码     //B 自己，从而实现互斥操作
    V(mutex);                           V(mutex);          //退出临界区
}                                   }
```

   4) 信号量解决同步问题：一组不同类进程，初始 cnt 为 0 或某个正整数，在**不同的进程**里分别执行 P，V 操作，实现通信，注意 **P，V 一定是成对的**；

单缓冲，单个生产者和单个消费者问题，需要信号量实现进程同步，表明缓冲区是满还是空；

```
semaphore empty,full;     //empty 指空闲缓冲区数量，full 指已填充缓冲的数量
empty =1;
full  =0;                 //初始状态缓冲区是空闲的
void Producer(){                        void Consumer(){
    while(true){                            while(true){
        getData();                              P(full);          //确认缓冲区不是空的
        P(empty);      //确认缓冲区有空闲       clearBuffer();
        fillBuffer();                           V(empty);         //通知生产者
        V(full);       //通知消费者             processData();
    }                                       }
}                                       }
```

大小为 k 的缓冲，多个生产者和多个消费者问题，需要信号量实现进程同步，表明缓冲区是满还是空；此外还要有一个互斥量防止生产者或消费者同时访问缓冲区；

```
semaphore empty,full,mutex;
empty =k; full  =0;             //empty 指空闲缓冲区数量，full 指已填充缓冲的数量
mutex =1;                       //mutex 实现互斥访问缓冲区
void Producer(){                        void Consumer(){
    while(true){                            while(true){
        getData();                              P(full);          //确认缓冲区不是空的
        P(empty);    //确认缓冲区有空闲         P(mutex);
        P(mutex);    //特别注意两者顺序，防止死锁 clearBuffer();
        fillBuffer();                           V(mutex);
        V(mutex);                               V(empty);         //通知生产者
        V(full);       //通知消费者             processData();
    }                                       }
}                                       }
```

理发师问题：1 位理发师，1 把理发椅，N 把供顾客等候的椅子；如果没有顾客，理发师就在理发椅上睡觉，有顾客到了，会唤醒理发师理发；如果理发师正忙，有顾客到了则坐在椅子上等，如果没有空椅子则离开；

```
const int RESTING_CHAIR_CNT=N;        //常量，供顾客等候的椅子的数量；
int waitingCustomerCnt=0;             //共享变量，正在等待的顾客的数量，需用 mutex 实现互斥访问；

semaphore barber, customer, mutex;
barber =0; customer =0;               //barber 指示理发师状态，customer 指示顾客状态
mutex  =1;
```

```
void Barber (){
    while(true){
        P(customer);  //确认有顾客，则开始理发
        P(mutex);     //特别注意两者顺序，防止死锁
        waitingCustomerCnt --;
        V(mutex);
        cutHair();
        V(baber);     //服务完一位顾客，通知其它
    }
}
```

```
void Customer (){
    P(mutex);
    if(waitingCustomerCnt<RESTING_CHAIR_CNT){
        waitingCustomerCnt ++; V(mutex);
        V(customer);  //通知理发师有顾客来了
        P(baber);     //确认理发师有空，等待理发
    } else {
        V(mutex);      //等候的顾客太多，离开
    }
}
```

5. 管程（Monitor）是把共享变量及其操作集中在一起统一控制的模块，提供了一种互斥机制；
6. 进程间的高级通信（IPC）主要用来传递数据，有共享内存；消息传递（直接通信，信箱）；管道（共享文件）；不同进程间的消息传递，由于内存地址空间的隔离，需通过操作系统才能完成；消息传递的同步方式通常采用非阻塞型 Send 和阻塞型 Receive；
7. 死锁：无论是相互通信的进程或是共享资源的进程，都可能因推进顺序不当或资源分配不妥而造成死锁；死锁是系统中一组并发进程因等待其它进程占有的资源，而永远不能向前推进的僵化状态；
8. 产生死锁的 4 个必要条件：**互斥条件、占有并等待条件、不剥夺条件和循环等待条件**；只要破坏这 4 个必要条件之一，就可以防止死锁；
9. 解决死锁问题的三种策略和方法：预防、避免、检测和解除；
   死锁的预防是系统**预先确定**一些资源分配策略，进程按此规定来申请和使用资源，保证死锁的某一个必要条件不会满足，使得系统不发生死锁，缺点是资源利用率低，或对资源使用的限制过严；
   死锁的避免涉及到**动态地分析和检测**新的资源请求和资源的分配情况，以确保系统始终处于安全状态，对资源的使用放宽了条件；**银行家行算法**是著名的死锁避免算法，但该算法要求预先获得有关信息，很少有进程能够在运行前就知道其所需的最大资源量，而且系统中进程数不是固定的，往往在不断地变化，所以该算法缺乏实用价值；
   若有m个资源，n个进程共享使用，每个进程需要的**最大**资源数为x，由银行家算法，$n(x-1)+1 \le m$时系统是安全的；若$m \le n$，则$x=1$；若$m>n$，则$x=1+(m-1)/n$；
   死锁的检测和解除算法中，操作系统总是同意资源申请，对资源的分配不加任何限制，也允许系统发生死锁，但必须建立一个检测机制，选择检测算法（如进程资源分配图、传递闭包等），周期性地检测是否发生了死锁，如果发生了，把它检测出来，再采取措施去解除死锁；虽然死锁检测对资源的使用不加任何限制，但开销比较大；

第四章、存储管理★

1. 存储管理的基本功能有：存储分配、地址转换（抽象和映射）、存储保护（隔离）和共享、存储扩充（虚存）；
   存储分配是给多道运行的作业分配主存空间；
   地址转换是为用户程序提供一个单一的，足够大的，连续的抽象逻辑地址空间，把逻辑地址空间中的用户程序通过静态重定位或动态重定位映射到分配的物理地址空间中，方便用户程序的编译和执行；
   存储保护指各道程序只能访问自己的存储区域，而不能互相干扰，以免程序受到有意或无意的破坏；存储共享指主存中的某些程序和数据可供不同用户进程同时使用；
   存储扩充是将辅存抽象为虚拟主存，允许用户进程的逻辑地址空间大于实际的物理内存容量；
2. 内存分配需要对空闲的内存进行管理，可采用 bitmap/数组或空闲链表的方法；
   最简单的单道系统中，如果一个程序能装入主存，那么它将一直运行，直到结束才返回操作系统；
   如果主存空间不足，可以通过**覆盖**或**交换**的技术解决；
   多道程序操作系统最简单的存储管理技术是分区方式，又分固定分区和可变分区；可变分区的**分配算法**有：最先适

应（空闲分区按地址排序，常用）、下次适应（从上次扫描结束处顺序查找最合适的空闲分区，可避免碎片集中在前端）、最佳适应（空闲分区按大小排序）、最坏适应、快速适应（常见大小分区单独管理，回收算法较复杂）；
伙伴系统是介于固定分区和可变分区之间的一种分配方案；
采用分区方式管理内存，每道程序总是要求占用主存的一个或几个连续存储区域，会产生许多碎片；如果主存空间不足，可以通过**拼接**（移动分区，合并碎片）、**覆盖**、**交换**技术解决；
现代计算机支持虚存，如请求分页式虚存管理，允许把一个进程的页面存放到若干不相邻接的主存页框中，消除了外碎片，但每个进程存在不多于一页的内碎片；

3. 虚拟存储器基于程序的局部性原理，不把一个用户进程的全部信息同时装入主存，而是仅将其中当前使用部分先装入主存储器，其余暂时不用的部分先存放在作为主存扩充的辅存中，待用到这些信息时，再由系统把它们装入到主存中，简而言之，它采用了自动的部分装入、部分对换、主存辅存独立编址但统一使用的技术，使得进程的虚拟地址空间可以远远大于物理内存容量，为用户编程提供了极大方便；通过虚拟存储器，所有的用户程序和数据都给出逻辑地址，在运行时由系统转换成物理地址，即动态地址转换；于是就允许一个进程位于主存的任何地址，它的位置还可以动态改变；
   分页式虚存管理实现页面共享较复杂；
   多级页表：减少页表本身占用的存储空间，增加了访存次数；
   快表：页表的高速缓存，减少地址映射的访存次数，全相联存储；
4. 反置页表：主要是为了解决 64 位系统中页表过大的问题；传统的页表，以逻辑页号为序号（无需存储），页表内容为对应的物理页号，适应逻辑页号→物理页号单方向的转换，每个进程有自己独立的一张页表；而反置页表是整个系统只有一张页表，以物理页号为序号（无需存储），页表内容为逻辑页号，为区分不同进程，还需记录进程 PID；为实现逻辑页号→物理页号的转换，需根据 PID 和逻辑页号按内容查找整个反置页表，为提高效率，可为系统增加一个 Hashtable，按 PID 和逻辑页号计算 hash，然后得到物理页号；
   反置页表中只包含已经调入内存的页面，并未包含未调入内存的页面，因而必须为每个进程建立一个外部页表，形式与传统页表一样，一般放在辅存，缺页时使用；
   反置页表实现共享页面较复杂；
5. 段式虚存是为满足现代高级程序设计语言二维地址的要求而引进的，按用户应用中逻辑上有完整意义的内容进行分段；为了实现段式存储管理，需要建立**每个进程的段表**，记录用户逻辑段与主存中物理段的对应关系，段式存储管理主存的分配与回收跟可变分区方式类似，可以通过直接分配、移动分配或调出分配来完成；
   如果在分页式存储管理的基础上实现分段式存储管理，就是段页式虚存管理；
6. 分页涉及以下的问题：
   页面装入策略：决定页面何时被装入主存，有请求式和预调式 2 种；
   页面清除策略：决定**修改过的**页面**何时**被回写到磁盘上，有请求式（被换出时）和预约式（进程自主决定）2 种；
   页面分配策略：根据进程生命周期中分给的页框数可否改变，分为固定分配和可变分配；
   页面替换策略：当内存中没有空闲页面时，需将已调入的若干页面写入辅存，腾出空闲页；根据作用范围是整个系统还是局限于某个进程内，分为全局页面替换算法（不区分进程）和局部页面替换算法（某个进程）；
   固定分配策略通常和局部替换策略配合使用；可变分配策略既可以配合全局替换策略，也可以采用局部替换策略；
7. 全局页面替换算法：
   1) 最佳页面替换算法 OPT，淘汰以后不再使用的页（向后看）；理论上最优，无法实现，用于评价其它算法；
   2) 先进先出 FIFO，淘汰最先进入的页面（亦即驻留时间最长的），可能出现 Belady 异常（增多可用页面**可能**反而使缺页率上升）；因为驻留时间长的页面很可能是频繁使用的页面，被替换掉后，可能还会用到，导致缺页；
   3) 第二次机会算法 SCR：FIFO 的改进，利用引用位，如果引用位为 0，说明没有使用过，将其替换掉；如果为 1，说明使用过，不替换此页，并将引用位置为 0，并移至队尾，再给一次机会；继续扫描，寻找可替换页；如果所有页面的引用位都为 0，那么第二遍扫描就会把队首的页面替换掉，这相当于 FIFO；
   4) 时钟页面替换算法 CLOCK：将第二次机会算法的队列改为**循环**队列，淘汰页面时，从指针当前指向的页面开始循环扫描队列，如果引用位为 1，则置 0 并跳过此页；如果引用位为 0，则将该页淘汰，并且指针推进一步；
   5) 最近最少使用 LRU：淘汰一段时间内最久未使用过的页面（向前看），是基于时间局部性原理；需建立页面队列，每使用一次页面就要调整队列的顺序，队首总是最近最少使用的页，从而被淘汰；模拟方法：a)淘汰引用位为 0 的页面，又称最近未使用页面替换 NRU；b)计数器记录使用次数，淘汰最少的页，又称最不经常使用页面替换 NFU；c)计时法，每次使用页面刷新绝对时间戳（值是递增的），淘汰时间戳最小的（即最近没有使用过的）页面；以上三种方法都要定时清零，而且多需增加硬件支持；
   6) LRU 性能较优，不会出现 Belady 现象；CLOCK 算法性能接近 LRU，且容易实现；

8．抖动现象指因页面替换算法不佳，页面被淘汰不久后，需要被调入使用，不久之后又被调出，如此反复的现象；
9．局部页面替换算法：
1) 局部最佳页面替换：理论分析用；
2) 工作集模型和替换算法：工作集指某一时间段内进程运行所需使用页面的集合，是程序局部性的近似表示，确定进程常驻集的大小，使得进程缺页率低，解决抖动现象，又不至于浪费内存；工作集大小随时间变化；持续跟踪页面变化较困难，选取合适的时间段（工作集窗口）也是一个难题；
3) 模拟工作集替换：a)老化算法，引用位+老化字段；b)引用位+时间戳；
4) 缺页频率替换 PFF：若相邻两次缺页时间超过临界值 t，则移除所有未使用过的页，缩小工作集，反之则增大；
10．随机替换算法也不失为一种简单有效的方案；

第五章、设备管理

1．设备和 CPU 之间的数据传送控制方式主要有 4 种：程序轮询方式、中断控制方式、DMA 方式和通道方式；
程序询问方式和中断控制方式仅适用于配置少量外设的场合，前者浪费 CPU 时间，设备与 CPU 只能串行工作；后者虽改进了上述缺点，但中断次数太多，CPU 累计用在处理中断上的时间很可观，并行操作的设备数量受中断处理速度的限制；
DMA 和通道方式较好的解决了上述问题，它们均采用了设备与和主存直接交换数据的方法，仅当一块数据传送结束，这两种方式才发出中断信号请求 CPU 干预，把 CPU 从繁杂的 IO 事务中解放出来；它们的区别是：DMA 要求 CPU 执行设备驱动程序启动设备，并做好传送数据的有关准备工作；通道则完全是一个相对独立的 IO 控制系统，当 CPU 发出 IO 启动命令后，它便接收控制权，完成全部 IO 操作；
2．IO 软件分 4 层：**设备中断处理程序、设备驱动程序、与设备无关的 IO 软件、用户层 IO 软件**；
设备中断处理程序通常是设备驱动程序的组成部分，位于底层，主要工作有：分析中断类型，作出相应处理，检查和修改进程状态等，它的任务要尽量少，以提高性能；
设备驱动程序中包括了所有**与设备相关**的代码，它的工作是：把用户提交的逻辑 IO 请求转化为物理 IO 操作的启动和执行，如设备名转化为端口地址、逻辑记录转化为物理记录、逻辑操作转化为物理操作等；它对上层的软件屏蔽所有的硬件细节；
与设备无关的 IO 软件的基本功能是执行适用于所有设备的常用 IO 功能，并向用户层软件提供一个一致的接口，如设备命名、设备保护、缓冲管理、提供与设备无关的存储块尺寸、设备分配和状态跟踪、错误处理和报告等；
用户层 IO 软件包括在用户空间运行的 IO 库例程和 SPOOLING 程序；
3．通道又称为 IO 处理机，通道技术实现了 IO 的独立性，及设备与 CPU 的并行性，大幅度提高系统整体性能；具有通道的计算机系统，IO 程序设计涉及 CPU 执行 IO 指令，通道执行通道命令，以及 CPU 和通道之间的通信；
与通道有关的若干概念，包括：CPU 的 IO 指令、通道的通道命令、通道程序、CCW，CAW 和 CSW（通道控制字，地址字和状态字）、IO 主程序和通道程序的编写、通道的启动和 IO 操作的执行等；
4．缓冲主要用于匹配设备和 CPU 的处理速度，还可以减少中断次数；IO 的一个重要特点是使用缓冲区；常用的缓冲技术有：单缓冲、双缓冲、多缓冲（循环缓冲，缓冲池）；缓冲区是由 IO 实用程序处理的，**而不是**由应用进程控制；
高速缓存 Cache 与缓冲 Buffer 的区别：Cache 内的数据是低速存储设备上数据的一部分，相当于备份，而 Buffer 内的数据一般是独有的，不一定有备份；若 Cache 缺失，可以直接访问低速存储，但 Buffer 是数据的必经之路；
5．为了提高磁盘 IO 的性能，采用了**磁盘驱动调度**和**磁盘缓冲区**；此外还有优化数据块分布方法；
常用磁盘驱动调度算法有：先来先服务 FCFS、电梯调度（来回扫描，没有就掉头）、最短查找时间优先 SSTF（可能饥饿）、扫描 SCAN（来回扫描，一定到头，可能饥饿）、分步扫描（分组处理，防止独占）、单向扫描（单向扫描，到头复位）等调度算法；
磁盘缓冲区通常在主存开辟，作为磁盘块在磁盘与其余主存之间的高速缓冲；由于程序局部性原理，磁盘缓冲区的使用可大幅度减少磁盘与主存之间 IO 传送的次数；**提前读，延迟写，RAM 盘**；
6．独立磁盘冗余阵列 RAID 采用一组较小容量的、独立的、可并行工作的磁盘驱动器组成阵列来代替单一的大容量磁盘，再加上冗余技术，数据能用多种方式组织，分布存储；能并行对多个磁盘读写，改进了 IO 性能和系统可靠性；
1) RAID 0：并行交叉存取（条带），速度快，但无校验，只要有一个磁盘损坏，便会造成数据丢失，可靠性差，故较少使用；
2) RAID 1：磁盘镜像，可靠性高，但磁盘容量的利用率只有 50%，读写速度相当于一块磁盘；
3) RAID 1+0：或称 RAID 10，用 4 块硬盘，先做镜像，再做条带，结合了 RAID 0 和 RAID 1 的优点；
4) RAID 2：RAID 0 的改进，增加了海明码校验，技术复杂，很少使用；
5) RAID 3：与 RAID 2 相似，但使用奇偶校验码，并用单独的一块磁盘来存储校验码，这块校验盘可能成为瓶颈；

6) RAID 4：与 RAID3 相似，但以块为单位校验；
7) RAID 5：校验码分布在各磁盘上；速度和可靠性都有提高，且得到较好的平衡，较常用；
8) RAID 6：双重校验措施，可靠性最高，写入速度差，较少应用；

7．SPOOLING 系统能把一个物理设备虚拟化成多个虚拟（逻辑）设备，用共享设备来模拟独享设备；在中断和通道的支持下，操作系统采用多道程序技术，合理分配和调度各种资源，实现联机的外围设备同时操作；
SPOOLING 系统主要由预输入、井管理和缓输出组成，已被用于打印控制和电子邮件收发等许多场合；
缓存在内存，井在硬盘；

第六章、文件管理★

1．文件系统把磁盘的硬件特性和用户隔离开来，为用户提供“按名存取”的功能，是操作系统中负责存取和管理信息的模块，用统一的方式实现用户和系统信息的存储、检索、更新、共享和保护，为用户提供方便有效的文件使用和操作方法；
实现按名存取关键在于解决文件名和物理地址的转换；文件系统将磁盘抽象成容纳一组命名文件的设备；
2．文件是由文件名标识的一组信息的集合，它的逻辑结构有流式文件和记录式文件；文件的物理结构是指文件在物理存储空间中存放方法和组织；文件的存储结构涉及块的划分、记录的排列、索引的组织、信息的搜索等许多问题，直接影响文件系统的性能；
3．文件的物理结构有：顺序文件，链接文件，**索引文件**，直接文件（hash）；
4．文件目录是实现按名存取的主要数据结构，文件系统的基本功能之一就是负责文件目录的建立、维护和检索，要求编排的目录便于查找、防止冲突，目录的检索方便迅速；实际的操作系统常使用树型目录结构；
目录可以按不同方法组织，有的把文件名、文件属性、磁盘地址放在一起组成 FCB 存放；有的仅存放文件名和 inode 号，其它信息则放到文件 inode 中；
5．文件共享是指不同用户（进程）共同使用同一个文件，文件共享不仅为不同用户完成共同的任务所必须，而且还可以节省磁盘空间，减少文件复制；文件共享有静态共享（硬链接，按 inode 链接，只能链接文件，不能跨文件系统）、动态共享（进程间共享）、符号链接共享（软链接，按文件名链接，可跨文件系统）；
6．磁盘空间管理：连续分配；非连续分配；空间管理方法：bitmap，空闲区表，空闲块链，成组空闲块；
7．内存映射文件和虚拟内存系统两种特殊类型的文件；
内存映射文件把进程的虚地址空间与某一个磁盘文件关联起来，使得进程对文件的存取转化为对关联内存区的访问，消除两者操作的差异，简化编程；
虚拟文件系统是为了让一个操作系统同时支持多种文件系统，对多种具体文件系统的共性进行抽象，定义一致的接口，而形成的一个与具体文件系统实现无关的虚拟的通用文件系统；

第七章、操作系统的安全与保护

第八章、网络和分布式操作系统

计算机网络

《数据与计算机通信 第九版》William Stalling 电子工业出版社 ISBN 978-7-121-13925-3

第一部分．概述

第1章、数据通信、数据网络和因特网

第2章、协议体系结构，TCP/IP 和基于因特网的应用程序

第二部分．数据通信

第3章、数据传输

1．基带信号，宽带信号；**数据→信号→传输**；

2．Nyquist 带宽：无噪声情况下的理想数据传输速率 $C = 2B \cdot \log_2 M$，其中 C 的单位 bps（bit/s）；B 为带宽，Hz；M 为每个码元的离散电平数；在带宽 B 一定的情况下，只要 M 足够大，C 可以为任意大，这当然是一种理想情况，实际上噪声的存在使 M 是有限的；

3．Shannon 容量公式：有噪声情况下的$C = B \cdot \log_2(1 + SNR)$； SNR=S/N，为信噪比；该公式中的噪声为白噪声，没有考虑冲激噪声和时延失真，因此实际能达到的数据率比理论值要低得多；

第4章、传输媒体

1．导向传输媒体：1）双绞线；2）同轴电缆；3）光纤，多模突变，多模渐变，单模；

2．无线传输：1）天线；2）地面微波（中继）；3）卫星微波；4）无线电广播；5）红外线

3．无线传播路径：从低频到高频，为 1）地波；2）天波（高层大气的电离层反射，实际为折射）；3）视距；

4．视距传输的损伤：1）自由空间损耗；2）大气吸收，主要是水蒸汽和氧气；3）多径，收到同一信号的多个不同时延的副本；4）折射，随着海拔上升，空气稀薄，无线信号折射向下弯曲；

第5章、信号编码技术

1．数字数据→数字信号：

数据率 bps，数据传输的速率，记为 R，则每个 bit 持续的时间为 1/R；

调制速率指信号电平改变的速率，取决于编码特性，又称波特率 Baud，记为 D，则$\mathrm{D = R/L = R/\log_2 M}$，其中 L 为每个信号元素中的 bit 数，M 为**不同**的信号元素**个数**，$\mathrm{L = \log_2 M}$；

信号的编码应尽量将传输功率集中在传输带宽的中心位置；

1）不归零电平 NRZ-L，0=高电平，1=低电平；有直流分量，多个相同码元传输时难以保持同步；

2）不归零 1 制 NRZI，0=起始位置无跳变，1=有跳变；是差分编码；

3）双极性 AMI，0=没有线路信号，1=正或负电平，连续的 1 则正负电平不断交替；

4）伪三进制码，1=没有线路信号，0=正或负电平，连续的 0 则正负电平不断交替；

5）曼彻斯特编码，0=中间位置从高向低跳变↓；1=中间位置从低向高跳变↑；双相位编码；

6）差分曼彻斯特编码，中间位置总有跳变，0=起始位置有跳变；1=起始位置无跳变；

7）双极性 8 零替换，高密度双极性 3 零，都属于扰码技术，用违规码替换掉连续的多个 0，增强同步能力；

2．数字数据→模拟信号：

1）振幅键控 ASK；2）频移键控 FSK，双值频移键控 BFSK，多值 MFSK；3）相移键控 PSK，双值 BPSK，差分 DPSK，四值 QPSK，正交（Orthogonal）QPSK；4）正交调幅 QAM，同频但相差 90°的两个载波，作为两个通道，分别经 ASK 调制不同的信号；

3．模拟数据→数字信号：

1）脉冲编码调制 PCM；2）非线性 PCM；3）增量调制 DM，delta modulation；

数字信号与模拟信号相比，优点为使用转发器而不是放大器，不存在噪声累积；采用时分复用 TDM，而模拟信号采用频分复用 FDM，TDM 不存在交调噪声；便于数字化处理；

4．模拟数据→模拟信号：

1）调幅 AM；2）调频 FM；3）调相 PM；FM 和 PM 都是角度调制；

第6章、数字数据通信

1．异步传输（按字符），同步传输（按帧）；

2．差错检测：1）奇偶校验；2）循环冗余检验 CRC；

3．差错纠正：差错检测和重传不适合无线应用，因为差错率较高，对卫星链路，传播时延很长，单帧也很长，重传的效率低，所以需要能直接纠正错误的编码方式；海明码，编码增益；
4．全双工和半双工；

第7章、数据链路控制

1．数据链路控制协议提供流量控制，差错检测和差错控制等功能；流量控制使**接收端**能够调整来自发送端的数据流，防止接收端缓存溢出；差错控制是通过重传损坏帧来实现的，这些帧可能是没有被确认，也可能是对方要求重传；
2．链路的 bit 长度$B = R \cdot d/V$，B 是以 bit 为单位的链路长度，即当 bit 流完全占满整个链路时的数量，R 为数据率，bps，d 为链路长度，m，V 是信号传播速率，m/s；
3．流量控制
   1）停止等待流量控制：源点必须待收到确认帧后才能继续发送下一帧；
   2）滑动窗口流量控制：全双工链路间，每个帧都有序号标识，发送/接收双方各维护一个窗口，两者窗口大小可以不同，但一般是相同的，且为 2^k-1，发送端窗口内是可以发送的帧，或已经发送，尚未确认的帧（这些帧可能重传，所以需要保存着）；只有当收到的帧的序号在接收窗口内，接收端才收下该帧，发送确认帧，并将接收窗口前移一帧；
4．可靠传输可通过确认（单独确认帧或捎带）和超时重传两种机制来实现；超时可应对帧丢失而未到达接收端，或接收端检测到帧错误，直接丢弃不发送确认，或接收端发送的确认帧丢失的情况；
5．差错控制，自动重传请求 ARQ：
   1）停止等待 ARQ，基于停止等待流量控制，增加了超时重传，需要标识区分重传帧；
   2）返回 N 帧 ARQ，基于滑动窗口流量控制；发送端收到一个确认帧，则说明该帧及之前的帧（不管有没有对应的确认帧）也都被收到了，即累积确认；接收端若发现某帧出错，则丢弃该帧及后续帧，或者发送一个拒绝帧，或者等待超时；发送端重传该帧及后续帧，直到该帧被正确收到；接收端收到的帧总是有序的，只需要一个帧的缓存窗口；
   3）选择拒绝 ARQ，被重传的只有接收到拒绝帧或超时的帧，减少了被重传的帧的数量；但接收端必须有足够大的缓存，以保存拒绝帧后续的帧，直到重传成功，并将这些帧按顺序传给上层；
6．高级数据链路控制 HDLC，同步传输，数据为帧形式，面向 bit 流；帧的两端有标志字段（若干独特的 bit 序列，0111 1110）起界定作用（上一帧的首和下一帧的尾可以共用一个标志字段）；帧中的某段数据如果与标志字段相同，需要被改写，可采用 bit 填充法：如果有 5 个连续的 1，则在后面插入一个 0，接收端在检测到连续的 5 个 1，且后面的一个是 0，会自动删除这个 0，从而恢复数据；这种方法实现了所谓数据透明性；

第8章、复用

1．频分复用 FDM，ADSL；波分复用（光的频分复用）；
2．时分复用 TDM；同步时分，统计时分；

第9章、扩频

1．扩频，跳频，直接序列扩频，码分多址（CDMA），频率和时间都复用，使多个用户间很少干扰地独立使用相同频带；
2．码片向量（-1，+1…），表示 1，按位取反则表示 0；不同站的码片向量正交；

第三部分．广域网

第10章、电路交换和分组交换

1．电路交换技术用于公用电话网络，在两个通信站点间建立一条专用的通路，在该连接存在期间，网络内的交换和传输资源完全为该电路的使用而保留；
2．分组交换是为了在数据通信量突发时提供比电路交换更高效的设施而设计的。站点以称为分组的小数据块形式传输数据，每个分组由用户数据和控制信息组成；PS：报文交换是介于上述两者之间的一种方案；
3．分组交换网络内部操作是数据报或虚电路
   1）数据报的各分组独立对待，终点相同的数据报可能沿不同的路由前进；
   2）虚电路在两个端点之间定义了一条路由，该虚电路的所有分组都沿着这条相同的路由前进；适合长报文；

第11章、异步传输方式

1．ATM 是**面向连接的**分组交换技术，它设计来提供类似电路交换网络的性能，同时又提供分组交换网络的灵活性和效率；它采用小的，固定长度的分组，称为信元 cell；ATM 现多被 IP 技术（无连接的）替代；
2．虚通道连接 VPC，包含多个具有相同端点的虚通路 VCC，而 VCC 是 ATM 中的逻辑连接；
3．ATM 服务种类：实时—恒定比特率，可变比特率；非实时—可变比特率，可用比特率，不指明比特率，保证帧速率；

第12章、交换网络中的路由选择

1．固定式路由选择策略，路由表，每个结点只要知道到达某一终点所需路由的下一个结点即可；

2．洪泛式，分组被发送到该结点的所有输出链路，为避免数据无限增长，一种办法是让每个结点记录已转发过的分组，当副本再次到达时则直接丢弃；另一种更简单的方法是在分组中增加一个跳数计数字段，每传递一次该计数减 1，当变成 0 时丢弃该分组；稳健性高，可用于发送紧急报文（甚至网络损坏严重）；还可用于虚电路路由的建立；以及广播的信息；缺点是通信负荷过高，此外还有数据泄露的安全风险；

3．随机路由；

4．自适应路由：路由选择的判断随网络结构的变化而变化；

1）**距离-向量路由算法**—所有结点都定期地将它们的整个路由表传送给所有相邻结点，路由表包含每个终点及其路径的距离，RIP 协议里该距离是路由器跳数；

2）**链路状态路由算法**—要求参与算法的**每个结点有完全的网络拓扑信息**，定期测试其所有相邻结点的状态和时延，如果发生显著变化，则采用洪泛法传播给所有结点，其它结点可用 Dijkstra 算法更新路由；简单地根据一次测定的时延来改变路由，在网络负载较重时可能导致路由在重负荷结点和相邻的轻负荷结点之间反复振荡，影响性能，可修改时延的计算方法，采用均值等方法平滑变化，减少振荡；OSPF 算法；

5．最小生成树算法：Dijkstra 算法，必须知道完整的网络拓扑；Bellman-Ford 算法，只涉及相邻结点，逐步扩散；

第13章、数据网中的拥塞控制

1．当通过网络传输的分组数量接近网络的处理能力时就会出现拥塞，这时网络的性能可能出现急剧下降；

2．拥塞控制：反压（利用拥塞的向前扩散性）；阻流分组（如因特网控制报文协议 ICMP 的“源点抑制”）；隐式拥塞信令（源点根据网络的时延来判断是否发生拥塞）；显式拥塞信令；

3．发生拥塞时，结点可简单的丢弃所有收到的分组，也可以通过某种流量控制措施降低网络的流量；

第14章、蜂窝无线网络

1．蜂窝无线网络使用了多个低功率的发送器，其覆盖范围被划分多个六边形平铺的蜂窝状；主要技术问题是衰落；

2．增大蜂窝网络容量的方法：1）添加新信道；2）频率借用或各蜂窝动态指派频率；3）蜂窝分裂；4）使用定向天线将蜂窝扇区化；5）微蜂窝；

第四部分．局域网

第15章、局域网概述

1．局域网 LAN 由**共享的**传输媒体，一组与媒体的接口，以及规范媒体接入的硬件和软件组成；

2．LAN 的拓扑结构有：环形，总线形，树形和星形（如果星形拓扑 LAN 的中心结点是集线器，则是以广播模式工作的，逻辑上仍是总线形的）；

3．虚拟局域网 VLAN，用**三层**交换机或**路由器**划分逻辑局域网（子网），缩小广播域；

第16章、以太网

1．CSMA，随机接入（令牌环网是受控接入）；1-坚持，空闲立即发送，总是持续监听；p-坚持，空闲时以概率 p 发送，以概率 1-p 延迟后再监听；0-坚持，空闲立即发送，否则随机延迟后监听；

2．百兆以太网是带冲突检测的载波监听多点接入 CSMA/CD 式的，采用半双工模式；先听后发，边听边发，冲突停发，随机重发；

IEEE802.3 和以太网都采用 1-坚持，**二进制指数退避**：当站点连续多次遇到冲突时，它仍不断重发，前 10 次重发的随机时延平均值会逐次加倍，如果还是遇到冲突，再做 6 次尝试，这时的平均时延保持不变，16 次重发失败后将放弃并报告有差错发生；

帧要足够长，才能在帧传输结束前检测到冲突（比正常传输电压更高的突变），如果帧的长度太短，检测不到冲突的发生，就相当于只是效率较低的 CSMA；

3．由于 LAN 的信道质量较好，故采用无连接的工作方式，不对数据帧编号，也不需发送确认帧，以简化系统，即实现的是尽最大努力交付的不可靠服务；差错的纠正由传输层的 TCP 协议来完成；

4．如果 CSMA/CD 以太网的两个站点距离很远，信号电压衰落很大，即便发生冲突，也不会超过检测阈值，导致无法检测到冲突，这使得以太网的距离不能太长；

5．争用期是以太网端到端的**往返**时延，用 2τ 表示，又称冲突窗口或碰撞窗口，只有在 2τ 时间内没有检测到冲突，才能肯定这次发送没有发生冲突；以太网规定争用期长度为 51.2μs，对 10Mbps，争用期可发送 512bit，即 64 B，如果前 64 B 没有发生冲突，说明成功抢占信道，那么此帧后面的数据也不会发生冲突；64 B 是以太网的最小帧长，如果数据+首部的总长小于 64 B，则需填充；小于 64 B 的帧都是无效帧；

6．千兆以太网（1Gbps，10Gbps）是交换式的；各结点都接到中心的**交换机**上，并采用**全双工**模式，实际上每个站点都是一个独立的冲突域，各站点间不会再产生冲突，不过一般仍采用 802.3 的 MAC 帧格式，也可以继续采用 CSMA/CD；

第17章、无线局域网

1．在无线局域网 WLAN 中进行冲突检测是不实际的，无线信号的强度变动范围很大，所以采用了冲突避免的方式，即 CSMA/CA，还增加了确认机制；

第五部分．网际协议和运输协议

第18章、网际协议

1．互联网包含多个独立的网络，并通过路由器互相连接；网际互连中使用最广泛的是网际协议 IP，IP 在高层数据报（如 TCP）上添加一个首部，以形成 IP 数据报，或称互联网协议数据单元 PDU；IP 协议是无连接的；

2．IPv4，子网（将主机号进一步划分为子网号+主机号），子网掩码（掩盖实际主机号，保留网络号和子网号）；

A 类 1~126	0	网络/7 位			主机/24 位	
B 类 128~191	1	0	网络/14 位		主机/16 位	
C 类 192~223	1	1	0	网络/21 位		主机/8 位
D 类 224~239	1	1	1	0	组播	
E 类 240~255	1	1	1	1	留用	

255.255.255.255 是本网络的广播地址，称为受限广播；0.0.0.0 是本主机；
网络号全 0，特定主机号的 IP 是本网络的特定主机；网络号 127，主机号不全为 0 或 1 的地址是本机环回地址；
特定网络号，主机号全 1 是外部路由器发给此网络的全部主机的广播，称为直接广播；
特定网络号，主机号全 0 是该特定网络的地址，不是一个实际有效的 IP 地址，不能作为源或目的地址；

3．网际控制报文协议 ICMP，提供了从路由器及其它主机到达某个主机的报文交付手段，提供的是对通信环境中遇到的有关问题的反馈信息，如差错报告报文（终点不可达，超时报文，参数问题，源点抑制，重定向），询问报文（有回送请求与应答，时间戳与应答，地址掩码请求与应答，路由器询问和通告）；ping 和 tracert 是**应用层**直接使用**网络层** ICMP 的例子，它们没有经过传输层的 TCP 或 UDP；

4．地址解析协议 ARP，采用广播询问-应答的方式实现 IP 地址到 MAC 物理地址的转换；

5．IPv6 在 IPv4 基础之上的改进有：1）扩展的 128 位地址空间；2）改进的选项机制，选项放在首部与运输层首部之间，多数不需路由器处理，简化了路由器的工作；3）地址自动设置，增加寻址的灵活性，支持资源分配；

6．VPN 和 IPSec;

7．本地地址转换 NAT，将专用网内部地址转化成有效的公网地址，使得整个专用网只需要一个公网 IP，大大节省了 IP 消耗；在 NAT 环境下，只有专用网内部主机才能发起通信；专用地址包括

1）10 . 0. 0. 0~ 10 .255.255.255（1 个 A 类网络）；
2）172. 16. 0. 0~ 172. 31.255.255（16 个 B 类网络）；
3）192.168. 0. 0~ 192.168.255.255（256 个 C 类网络）；

8．无分类编址 CIDR 消除了 A，B，C 类网络地址的划分方法，而采用“IP 地址/网络前缀位数”的记法；

第19章、互联网的操作

1．组播（或称多播，为 1 对多通信，但不一定是全体，只是运行组播协议的端点；与广播不同，广播是 1 对全部端点通信），使用 D 类地址（并非所有 D 类都可以作为组播地址），只能作为目的地址，不能作为源地址，尽最大努力交付，不提供可靠交付，一定是仅应用于 UDP，不产生 ICMP 差错报告报文（即永远 ping 不通）；

1）组播地址可换算为 MAC 地址：取组播地址的后 23 位，并在高位补 0，在前面加上固定的首部 01-00-5E；
2）网际组管理协议 IGMP 用于主机和路由器经过 LAN 交换组播成员信息；
3）IP 组播路由的关键是建立一个组播分配树，数据沿着分配树传播；

2．自治系统；路由选择协议：距离向量路由选择—路由信息协议 RIP（16 跳为不可达，只能选择一条路由，使用 UDP 传送 RIP 报文，广播，应用层协议），链路状态路由选择—开放最短路径优先协议 OSPF（网络稳定时路由信息量少，以 IP 数据报组播方式发送协议报文，传输层协议）；

3．边界网关协议 BGP 是外部路由协议，允许不同自治系统中的路由器能够在交换路由选择信息时互相合作；边界路由器一般称为网关；BGP 报文通过 TCP 发送，应用层协议；BGP 采用路径-向量协议，路由表保存的是目标网络和其完整路径，该路径不一定是最优的；

4．移动 IP，将 IP 地址的标识唯一主机和作为路由地址的功能分开，移动终端的 IP 是固定的，在外地时（漫游）不用来作为路由，它需要一个归属网络的本地代理和一个外部代理；移动终端从外部代理得到一个转交地址（辅地址，

发现阶段），并通过一系列认证、注册，告知归属代理（登记阶段），然后归属代理通过**隧道**技术（IP 包在 IP 包中封装）实现移动终端与归属网络的通信；

第20章、互联网的服务质量

1．综合服务体系结构 ISA 是对因特网和内部网上种类和数量不断增长的通信量的响应。它提供了协议开发的框架，在基于 IP 的互联网上提供 QoS 支持，如开发 RSVP 协议来处理多媒体/组播通信量；ISA 的主要问题是在拥塞时如何共享可用的网络容量，主要是采用准入控制（预约，RSVP 协议），路由选择算法（不仅是时延，还采用其它参数来选择路由），排队规则（考虑不同流的不同需求），分组丢弃策略（选择丢弃哪些分组）；

2．互联网上传统的通信量是弹性的，而语音和视频等通信对实时性要求较高，是非弹性的；

3．区分服务 DS 体系结构被设计来提供简单的，容易实现的，低开销的工具，以支持许多网络服务，这些服务基于性能区分成多类，DS 基于 IP 首部里的 6 个 bit 标记，该标记将通信量按照路由器要给其提供服务的类型分类；

第21章、多协议标记交换

1．多协议标记交换 MPLS 是一组设计用来解决许多问题的协议，包括流量工程，QoS，VPN 和 IP-ATM 互连等；

2．MPLS 网络包括一组称为标记交换路由器 LSR 的结点，可以根据每个分组上附带的标记来交换分组和选择路由；MPLS 的关键要素是使用流量工程来优化和管理负荷；MPLS 很适合用来实现 VPN；MPLS 是面向连接的；

第22章、运输协议

1．运输协议提供了端到端的数据传送服务，它为上层协议屏蔽了网络的细节，运输协议可以是面向连接的，如 TCP，或者面向无连接的，如 UDP；

2．TCP 协议提供了连接建立和关闭，流量控制和拥塞控制的功能；

连接的建立采用“三次握手”，即分 3 步（注：确认序号 ackNo.是对端下一次将要发送的数据序号 seqNo.）：

1） 主机 A SYN=1， seqNo.=x

2） SYN=1，ACK=1，seqNo.=y， ackNo.=x+1 主机 B

3） 主机 A ACK=1，seqNo.=x+1，ackNo.=y+1

而连接的释放则需要 4 步，通信双方主机都可以发起释放连接的操作：

1） 主机 A FIN=1， seqNo.=u

2） ACK=1，seqNo.=v， ackNo.=u+1 主机 B 并通知本机的上层程序将要结束连接；

3） FIN=1，ACK=1，seqNo.=w， ackNo.=y+1 主机 B

4） 主机 A ACK=1，seqNo.=u+1，ackNo.=w+1；

主机 B 收到 A 最后的 ACK 后就关闭连接，而不再向 A 发 ACK；A 在等待 2*MSL（最大报文段生存时间）后关闭连接；

3．TCP 协议是**面向字节的**，每个字节占用一个序号；对端的确认序号 ackNo.是接收到数据的最高序号+1，即期望下次收到的数据的第一个字节的序号；IP 头 4 B 一个序号，IP 分片则是 8 B 一个序号；

4．TCP 流量控制：即让发送端的发送速率不致太快，让接收端来得及接收，也不致网络拥塞；利用滑动窗口机制，即接收端发送零窗口通知，发送端停止发送并设置计时，稍后发送零窗口探测报文，如果对方应答窗口不为 0，则继续发送，否则重置计时，持续探测；

5．TCP 拥塞控制：开环控制（事先考虑周到，力求避免），闭环控制（监测，反馈，调整）；

发送端需维护：1）接收端窗口值 Rwnd（接收端告知的）；2）拥塞窗口值 Cwnd（发送端估计的）；发送端的发送窗口值应取前面 2 个窗口值中较小的那个；

1）慢开始：刚开始设置拥塞窗口 Cwnd=1，每接收到一个新的 ACK 就将 Cwnd 加 1，实际 Cwnd 是指数增加的；

2）拥塞避免：当 Cwnd 超过了预先设置的门限值 ssthresh，停止使用慢开始算法，而是每经过一个往返时延 RTT，就将 Cwnd 加 1，即线性增长（此阶段不必再考虑 ssthresh 的限制）；

无论慢开始阶段还是拥塞避免阶段，只要发送端判断出现拥塞（没有按时收到 ACK），就把门限值 ssthresh 设置为出现拥塞时**发送窗口值**的 1/2（称为**乘法减小**，但必须≥2），并将 Cwnd 重置为 1，重新执行慢开始算法；这样就能迅速减少发送到网络中的分组数，使发生拥塞的路由器有时间处理积压的分组；

3）快重传算法：接收端**每收到**一个**失序**的报文段后就立即发出**重复确认**，让发送端及早知道有报文段丢失，发送端只要连续收到 3 个重复确认就应当立即重传对端**尚未收到的**报文段，而不是等到超时后再重传；

4）快恢复算法：若发送端连续收到 3 个重复确认，这时网络很可能**没有**发生拥塞，因为虽然发生了丢包，但毕竟多个重复确认发送过来了，故之后执行**乘法减小**，**但不是**继续执行慢开始算法（从 0 指数增加），而是执行拥塞避免算法（即将 Cwnd 设置为拥塞时的 1/2 后线性增加）；

第六部分. 网络安全

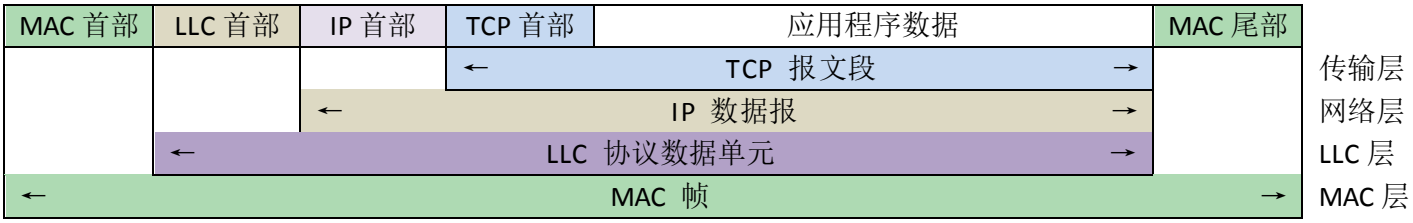
第23章、计算机与网络安全威胁

第24章、计算机与网络安全技术

OSI	TCP/IP	5 层参考模型	说明
应用层 表示层 会话层	应用层	应用层 （用户↔用户）	**任务：**提供系统与用户的接口； **协议：**FTP、SMTP、POP3、HTTP、SSH 等；
传输层 （有连接）	传输层 （有或无连接）	传输层 （进程↔进程）	**任务：**负责两个不同主机中进程间的通信； **功能：**为端到端连接提供 1.可靠的传输服务（TCP）；2.流量控制，差错控制，QoS 等管理服务； **协议：**TCP、UDP； **传输单位：**报文段（TCP）、用户数据报（UDP）；
网络层 （有或无连接）	网络层 （无连接）	网络层 （主机↔主机）	**任务：**1.将上层的报文段封装成分组；2.选择路由，交付分组； **功能：**1.为上层提供服务；2.组包，拆包；3.路由选择；4.拥塞控制； **协议：**IP（v4，v6）、ARP，RARP、ICMP、OSPF、RSVP 等； **传输单位：**IP 数据报；　　**硬件：**路由器；
数据链路层	**网络接口层**	数据链路层 逻辑链路层 LLC+ 媒体访问控制层 MAC	**任务：**将上层的 IP 数据报组装成帧； **功能：**1.链路连接的建立，拆除；2.帧界定，帧同步；3.差错检测； **协议：**PPP、HDLC、ARQ 等； **传输单位：**帧；　　**硬件：**网桥，交换机；
物理层		物理层	**任务：**透明地传输 bit 流； **功能：**提供数据通路； **传输单位：**bit；　　**硬件：**双绞线，中继器，集线器等；

注：ARP：将 IP 地址（v4 为 32 位）转换为 MAC 地址（物理地址，48 位）的协议；RARP 则提供反向的功能；
PPP：点对点协议；

根据 RFC894，以太网封装的 IP 数据包长度是 46 B ~ 1500 B；以太网 MAC 帧长为 64 B~1518 B；
（~~前导同步码 7B＋帧界定符 1B＋~~）目的 MAC 6B＋源 MAC 6B＋帧类型 2B＋ IP 数据报 46~1500 B＋帧校验序列 FCS 4B



设备	隔离广播域	隔离冲突域	说明	所在层
中继器（两端口） 集线器（多端口）	×	×	对线路上的信号放大整形；若连接 N 个设备则每个设备得到总带宽的 1/N；	物理层
网桥（两端口） 交换机（多端口）	×	√	根据 MAC 地址对信号转发，直通或存储转发；每个设备均得到总带宽；	数据链路层 （第二层）
路由器	√	√	连接多个网络，根据 IP 地址为信号选择路由；实际相当于一种专用计算机；	网络层 （第三层）

网桥/交换机的每个端口都是一个冲突域；而路由器的每个端口都是一个广播域；
一个广播域就是一个局域网（或虚拟局域网 VLAN）；
划分 VLAN 可以基于：1）三层交换机的端口；2）终端设备的 MAC 地址；3）IP 地址和子网掩码；4）更上层的协议；